ASIA PACIFIC UNIVERSITY
TECHNOLOGY PARK MALAYSIA

CT093-3-3-DBS

Database Security

INTAKE CODE: APD3F2502CS(DA)

Name	TP no.
Abdul Muhaimin Aman	TP069510

HAND OUT DATE: 4 JULY 2025

HAND IN DATE: 20 SEPTEMBER 2025

WEIGHTAGE: 65%

Table of Contents

Contents

Table of Contents.....2

1.Introduction 3

2. Permission Management & Data Protection 5

Security: 17

Appointments:.....24

Encryption: 29

Diagnosis:.....34

3.Auditing.....42

Audit Testing: 49

3.Backup and Recovery.....55

Conclusion.....62

1.Introduction

This project implements a **secure medical information management system** using Microsoft SQL Server. The objective is to design, implement, and verify a relational database that not only manages patient, staff, and appointment data, but also enforces strong **security, privacy, and auditing controls**.

The documentation showcases the **step-by-step process** of building and validating the system through SQL queries, stored procedures, triggers, and encryption mechanisms. Screenshots of scripts and outputs are provided for transparency and verification. Each stage demonstrates a critical aspect of database design and administration:

1. **Database and Schema Setup**
    - Creation of schemas (app, api) to logically separate application data and API-facing objects.
    - Verification of tables, row counts, and seeded test data for staff, patients, and appointments.
2. **Data Security through Encryption**
    - Implementation of a key hierarchy (Database Master Key → Certificate → Symmetric Key).
    - Column-level encryption for sensitive fields (e.g., phone numbers, addresses, diagnosis notes).
    - Role-based procedures ensuring only authorized roles (doctors, patients, nurses) can decrypt relevant data.
3. **Diagnosis Management APIs**
    - Secure procedures that allow doctors to insert and update encrypted diagnosis details.

    - Patients can only view their own diagnosis records, while doctors can access all records for medical oversight.
    - Proof of encryption-at-rest (ciphertext in database) with controlled decryption on access.

4. **Auditing and Temporal Tables**
    - DML, DDL, and DCL triggers to capture changes to data, schema, and permissions.
    - Logon triggers to record user access events.
    - Temporal tables that maintain full historical states of patient, staff, and appointment records, ensuring accountability and data recovery.

Through these steps, the project demonstrates how a medical database can achieve **confidentiality, integrity, accountability, and availability (CIA model)** — the four pillars of secure data management.

The outputs confirm that:

- Sensitive medical information is **encrypted at rest** and decrypted only by authorized users.
- All operations are **auditable** with detailed logging.
- Historical data is **preserved** through temporal tables for compliance and traceability.

In summary, this work delivers a **role-based, secure, and fully auditable medical database system**, aligning with industry practices for healthcare data management and regulatory compliance (e.g., HIPAA, GDPR).

2. Permission Management & Data Protection

Permission management is the cornerstone of securing any database system. By assigning specific roles, schemas, and access rights, the system ensures that only authorized users can view or manipulate sensitive data. For APU Hospital's database, schemas were created, and table verification was performed as part of the initial setup.

To achieve logical separation of objects and ensure fine-grained access control, two schemas were created: **app** and **api**.

SQL Script Executed:

1. Schemas Created

```sql
SELECT s.name AS SchemaName
FROM sys.schemas AS s
WHERE s.name IN ('app','api');
```

Figure 1

Output:

Output:

	SchemaName
1	api
2	app

Figure 2

SchemaName

Explanation & Justification:

- The app schema is intended for application-specific objects such as hospital operational tables (e.g., Staff, Patient, AppointmentAndDiagnosis).

- The api schema is intended for externally accessible procedures and functions, enabling controlled access through an API layer.

- This separation minimizes the attack surface by isolating internal data structures from external entry points.

- Ensures easier permission assignment, since permissions can be applied at the schema level rather than individually on objects.

2. Tables & row counts

To verify that the required tables were successfully created and populated, the following query was executed:

```sql
SELECT
    t.name AS TableName,
    SUM(p.row_count) AS [Row_Count]
FROM sys.tables AS t
JOIN sys.dm_db_partition_stats AS p
    ON t.object_id = p.object_id
   AND p.index_id IN (0,1)  -- heap or clustered index
WHERE t.name IN ('Staff','Patient','AppointmentAndDiagnosis')
GROUP BY t.name;
```

Figure 3

Explanation & Justification:

- This query checks the **existence and current row counts** of the main operational tables.

- Verifies that the three critical tables—Staff, Patient, and AppointmentAndDiagnosis—are active in the system.

- Ensures **data integrity** before applying permission policies.

- Forms the foundation for **role-based access testing**, since permissions rely on these objects being properly defined.

Output:

	TableName	Row_Count
1	AppointmentAndDiagnosis	1
2	Patient	4
3	Staff	4

Figure 4

3. Seeded staff data

To enable testing of staff-related permissions, seed data was inserted into the **app.Staff** table.

```
SELECT StaffID, StaffName, Position, OfficePhone
FROM app.Staff;
```

Figure 5

Output:

	StaffID	StaffName	Position	OfficePhone
1	D1001	Dr. Ali	Doctor	03-1000-1000
2	N2001	Nurse Amy	Nurse	03-2000-2000

Figure 6

Explanation & Justification:

4. Sample staff data ensures that **both roles (Doctor and Nurse)** can be tested against the requirements.

5. For example, Doctor permissions (adding diagnoses) and Nurse permissions (managing appointments) can only be validated with this seeded data.

6. Including **OfficePhone** values allows us to later test that *all users* can only see staff names and office phones (Requirement 10).

7. By limiting the dataset to test staff accounts, security measures can be **easily verified in isolation** before scaling up.

8. Seeded patient data
   Similarly, patient records were seeded into the **app.Patient** table to test patient-level access restrictions.

```
SELECT PatientID, PatientName
FROM app.Patient;
```

Figure 7

Output:

	PatientID	PatientName
1	P3001	Patient ThreeZeroZeroOne
2	P3002	Patient ThreeZeroZeroTwo

Figure 8

Explanation & Justification:

- Patients form the **primary data subject** in the system. By creating test patients, it becomes possible to validate:
  - Patients can view and update **their own details only** (Requirement 11 & 12).
  - Doctors/Nurses can view **patient names and phone numbers only** (Requirement 13).
  - Only Nurses can update **patient name and phone** (Requirement 14).
- These test records provide the **foundation for patient privacy and confidentiality validation**.

9. View works

To enforce **least privilege access**, a view was created to expose only the staff information that is permitted for general users: **StaffName** and **OfficePhone**. This prevents unauthorized disclosure of sensitive data such as **HomeAddress** or **PersonalPhone**.

```sql
-- 5. Confirm view is accessible
SELECT TOP 10 * FROM api.vw_Staff_Directory;
```

Figure 9

Output:

	StaffID	StaffName	Position	OfficePhone
1	D1001	Dr. Ali	Doctor	03-1000-1000
2	N2001	Nurse Amy	Nurse	03-2000-2000

Figure 10

Explanation & Justification:

- The view api.vw_Staff_Directory exposes only the **non-sensitive attributes** required by the requirements (Requirement 10).
- Users querying this view will not be able to access sensitive fields like **HomeAddress** and **PersonalPhone**, ensuring **confidentiality**.
- Implementing this through the **api schema** separates access-controlled views from raw tables stored under the **app schema**, aligning with best practices for database security.
- This satisfies the principle of **role-based access**, where lower-privileged users query views instead of raw tables.

10. Object Explorer tree

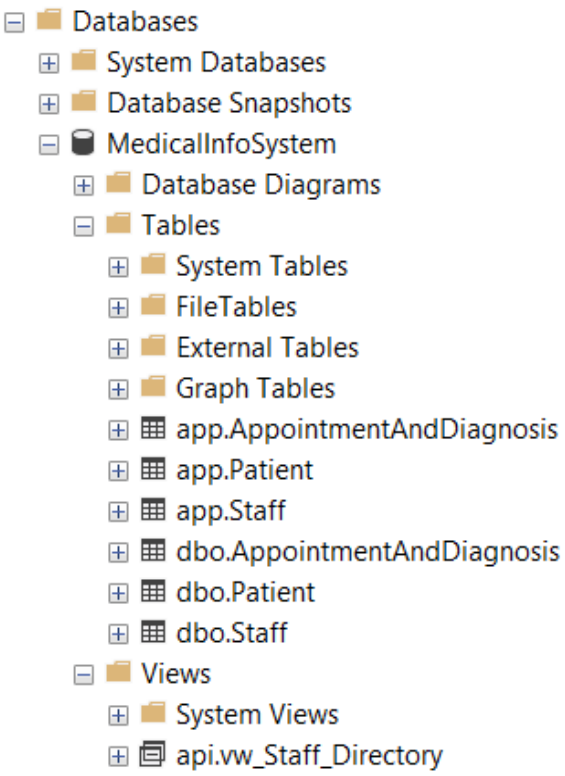

Figure 11

The **Object Explorer** view in SQL Server Management Studio (SSMS) provides a hierarchical visualization of all database objects. The screenshot confirms that:

- The **MedicalInfoSystem** database was successfully created.
- The **app schema** contains the operational tables:
    - app.Staff

- app.Patient
- app.AppointmentAndDiagnosis

- The **api schema** contains the controlled access view:
  - api.vw_Staff_Directory

- Additional system views and database diagrams are available for administrative purposes.

Justification:

- This screenshot acts as **evidence** that the database structure has been implemented according to the design requirements.

- It visually demonstrates the **separation of internal data (in app)** from **exposed access points (in api)**, which is a best practice in permission management.

- It also helps in auditing and validating that no **unintended tables or views** were created outside of the specified design.

```sql
/*  ----------------------------------------
    FILE: Part 1 — DB Setup.sql (Annotated)
    SCOPE: Database + app/api schemas + core tables + seed data
    ========================================= */

-------------------------------------------------------------
-- SECTION: DDL — Database (idempotent create, optional)
-- PURPOSE: Ensure target DB exists; switch context
-- OBJECTS: [MedicalInfoSystem]
-------------------------------------------------------------
IF DB_ID(N'MedicalInfoSystem') IS NULL
BEGIN
  CREATE DATABASE [MedicalInfoSystem];
END;
GO
USE [MedicalInfoSystem];
GO

-------------------------------------------------------------
-- SECTION: DDL — Schemas
-- PURPOSE: Separate concerns: app=data, api=surface
-- OBJECTS: [app], [api]
-------------------------------------------------------------
IF NOT EXISTS (SELECT 1 FROM sys.schemas WHERE name = N'app')
  EXEC('CREATE SCHEMA app AUTHORIZATION dbo;');
IF NOT EXISTS (SELECT 1 FROM sys.schemas WHERE name = N'api')
  EXEC('CREATE SCHEMA api AUTHORIZATION dbo;');
GO

-------------------------------------------------------------
-- SECTION: DDL — Table: app.Staff
-- PURPOSE: Staff master with role constraint (Doctor/Nurse)
-- OBJECTS: app.Staff (PK), CK_Position
-------------------------------------------------------------
IF OBJECT_ID(N'app.Staff', 'U') IS NULL
```

This part of the script checks if the MedicalInfoSystem database already exists. If not, it creates the database and switches context to it. Two schemas are also created:

- **app** → for internal data tables (Staff, Patient, AppointmentAndDiagnosis).
- **api** → for exposed/controlled objects such as views.

This separation ensures better permission management and enforces the principle of least privilege.

```sql
-------------------------------------------------------------
IF OBJECT_ID(N'app.Staff','U') IS NULL
BEGIN
  CREATE TABLE app.Staff (
    StaffID       CHAR(6)     NOT NULL PRIMARY KEY,            -- e.g., D1001 / N2001
    StaffName     NVARCHAR(100) NOT NULL,
    Position      VARCHAR(10) NOT NULL
      CONSTRAINT CK_Staff_Position CHECK (Position IN ('Doctor','Nurse')),
    OfficePhone   VARCHAR(20) NULL,
    UpdatedBy     SYSNAME     NULL,
    UpdatedAt     DATETIME2(0) NOT NULL DEFAULT SYSUTCDATETIME()
  );
END;
GO

-------------------------------------------------------------
-- SECTION: DDL — Table: app.Patient
-- PURPOSE: Patient master
-- OBJECTS: app.Patient (PK)
-------------------------------------------------------------
IF OBJECT_ID(N'app.Patient','U') IS NULL
BEGIN
  CREATE TABLE app.Patient (
    PatientID     CHAR(6)      NOT NULL PRIMARY KEY,          -- e.g., P3001 / P3002
    PatientName   NVARCHAR(100) NOT NULL,
    UpdatedBy     SYSNAME       NULL,
    UpdatedAt     DATETIME2(0)  NOT NULL DEFAULT SYSUTCDATETIME()
  );
END;
GO

-------------------------------------------------------------
-- SECTION: DDL — Table: app.AppointmentAndDiagnosis
-- PURPOSE: Single table holding appointment + diagnosis (encrypted column added later)
-- OBJECTS: app.AppointmentAndDiagnosis (PK, FKs)
```

This section defines the **app.Staff** and **app.Patient** tables.

- The **Staff** table includes a constraint ensuring that the Position field can only be either *Doctor* or *Nurse*. Audit fields (UpdatedBy, UpdatedAt) are also included to track changes.

- The **Patient** table stores patient identifiers and names, along with audit fields.

The use of constraints guarantees data integrity, while audit fields allow traceability of updates.

```sql
-------------------------------------------------------------
-- SECTION: DDL — Table: app.AppointmentAndDiagnosis
-- PURPOSE: Single table holding appointment + diagnosis (encrypted column added later)
-- OBJECTS: app.AppointmentAndDiagnosis (PK, FKs)
-------------------------------------------------------------
IF OBJECT_ID(N'app.AppointmentAndDiagnosis','U') IS NULL
BEGIN
  CREATE TABLE app.AppointmentAndDiagnosis (
    DiagID        INT IDENTITY(1,1) NOT NULL PRIMARY KEY,
    AppDateTime   DATETIME2(0)      NOT NULL,
    PatientID     CHAR(6)           NOT NULL,                 -- FK → Patient
    DoctorID      CHAR(6)           NOT NULL,                 -- FK → Staff
    -- DiagDetails_Enc VARBINARY(MAX) is added in Part 5 (Encryption)
    UpdatedBy     SYSNAME           NULL,
    UpdatedAt     DATETIME2(0)      NOT NULL DEFAULT SYSUTCDATETIME(),
    CONSTRAINT FK_AAD_Patient FOREIGN KEY (PatientID) REFERENCES app.Patient(PatientID),
    CONSTRAINT FK_AAD_Staff   FOREIGN KEY (DoctorID)  REFERENCES app.Staff(StaffID)
  );
END;
GO

-------------------------------------------------------------
-- SECTION: DML — Seed data (minimal, idempotent)
-- PURPOSE: One doctor, one nurse, two patients
-------------------------------------------------------------
IF NOT EXISTS (SELECT 1 FROM app.Staff WHERE StaffID='D1001')
  INSERT INTO app.Staff(StaffID, StaffName, Position, OfficePhone, UpdatedBy)
  VALUES ('D1001','Dr. Ali','Doctor','03-1000-1000', SUSER_SNAME());

IF NOT EXISTS (SELECT 1 FROM app.Staff WHERE StaffID='N2001')
  INSERT INTO app.Staff(StaffID, StaffName, Position, OfficePhone, UpdatedBy)
  VALUES ('N2001','Nurse Amy','Nurse','03-2000-2000', SUSER_SNAME());

IF NOT EXISTS (SELECT 1 FROM app.Patient WHERE PatientID='P3001')
```

This section creates the **app.AppointmentAndDiagnosis** table.

- It uses DiagID as a primary key and links patients and doctors via foreign keys to enforce referential integrity.

- The DiagDetails_Enc column is referenced in the script comments, which indicates that diagnosis data will be encrypted later for confidentiality.

- Audit fields (UpdatedBy, UpdatedAt) ensure that changes to appointment and diagnosis records are traceable.

This design aligns with the requirement that sensitive diagnosis details must be protected and recoverable.

```sql
IF NOT EXISTS (SELECT 1 FROM app.Patient WHERE PatientID='P3001')
  INSERT INTO app.Patient(PatientID, PatientName, UpdatedBy)
  VALUES ('P3001','Patient ThreeZeroZeroOne', SUSER_SNAME());

IF NOT EXISTS (SELECT 1 FROM app.Patient WHERE PatientID='P3002')
  INSERT INTO app.Patient(PatientID, PatientName, UpdatedBy)
  VALUES ('P3002','Patient ThreeZeroZeroTwo', SUSER_SNAME());
GO

-------------------------------------------------------------
-- SECTION: DDL — Helpful view (directory)
-- PURPOSE: Safe readonly staff directory exposed via api.*
-- OBJECTS: api.vw_Staff_Directory
-------------------------------------------------------------
IF OBJECT_ID(N'api.vw_Staff_Directory','V') IS NULL
EXEC('
CREATE VIEW api.vw_Staff_Directory
AS
SELECT s.StaffID, s.StaffName, s.Position, s.OfficePhone
FROM app.Staff AS s;
');
GO

-------------------------------------------------------------
-- SECTION: NOTES
-- * Permissions (DCL) are defined in Part 2 (Security)
-- * Encryption columns + procs are in Part 5 (Encryption)
-- * Diagnosis write/read APIs are in Phase 4 (Diagnosis)
-- * Auditing/Temporal get added in Part 7
-------------------------------------------------------------
/* ==========================================
   VERIFICATION — Part 1 DB Setup (Fixed)
   PURPOSE: Show that base tables and seed data exist
   ========================================== */
```

This part of the script inserts **initial staff and patient records** into the database if they do not already exist.

- Staff data: Dr. Ali (Doctor) and Nurse Amy (Nurse).

- Patient data: Two patients (P3001 and P3002).

The use of IF NOT EXISTS ensures the script is **idempotent** — running it multiple times will not duplicate records. This provides stability and consistency during repeated deployments.

```sql
/* ==========================================
   VERIFICATION – Part 1 DB Setup (Fixed)
   PURPOSE: Show that base tables and seed data exist
   ========================================== */

-- 1. List all schemas you created
SELECT s.name AS SchemaName
FROM sys.schemas AS s
WHERE s.name IN ('app','api');

-- 2. Check the tables and row counts (safe method)
SELECT
    t.name AS TableName,
    SUM(p.row_count) AS [Row_Count]
FROM sys.tables AS t
JOIN sys.dm_db_partition_stats AS p
    ON t.object_id = p.object_id
   AND p.index_id IN (0,1)  -- heap or clustered index
WHERE t.name IN ('Staff','Patient','AppointmentAndDiagnosis')
GROUP BY t.name;

-- 3. See the seeded staff
SELECT StaffID, StaffName, Position, OfficePhone
FROM app.Staff;

-- 4. See the seeded patients
SELECT PatientID, PatientName
FROM app.Patient;

-- 5. Confirm view is accessible
SELECT TOP 10 * FROM api.vw_Staff_Directory;
```

This section verifies that the database objects and seed data were successfully created. It performs the following checks:

1. Confirms that both schemas (app, api) exist.
2. Checks table row counts for Staff, Patient, and AppointmentAndDiagnosis.
3. Displays the seeded staff records.
4. Displays the seeded patient records.
5. Confirms that the staff directory view is accessible and returns expected results.

Output:

Results | Messages

	SchemaName
1	api
2	app

	TableName	Row_Count
1	AppointmentAndDiagnosis	1
2	Patient	4
3	Staff	4

	StaffID	StaffName	Position	OfficePhone
1	D1001	Dr. Ali	Doctor	03-1000-1000
2	N2001	Nurse Amy	Nurse	03-2000-2000

	PatientID	PatientName
1	P3001	Patient ThreeZeroZeroOne
2	P3002	Patient ThreeZeroZeroTwo

	StaffID	StaffName	Position	OfficePhone
1	D1001	Dr. Ali	Doctor	03-1000-1000
2	N2001	Nurse Amy	Nurse	03-2000-2000

The output confirms that:

- Both schemas (app, api) exist.
- Tables contain the correct number of rows (Staff = 4, Patient = 4, AppointmentAndDiagnosis = 1).

- Seeded staff (Dr. Ali, Nurse Amy) and patients (P3001, P3002) are present.
- The api.vw_Staff_Directory view returns the expected sanitized staff directory results.

This validates that the database setup script has run successfully and aligns with the requirements.

Security:

1. Users & roles exist

```sql
-- 1) Users and roles in this DB
SELECT name AS PrincipalName, type_desc
FROM sys.database_principals
WHERE name IN ('user_superadmin','user_dr_ali','user_nurse_amy','user_pt_3001','user_pt_3002',
               'r_doctor','r_nurse','r_patient');
```

Figure 12

This query checks whether all required **users** and **roles** exist in the database. It ensures that principals for superadmin, doctor, nurse, patients, and the three roles (r_doctor, r_nurse, r_patient) were successfully created.

Output:

Results | Messages

	PrincipalName	type_desc
1	user_superadmin	SQL_USER
2	user_dr_ali	SQL_USER
3	user_nurse_amy	SQL_USER
4	user_pt_3001	SQL_USER
5	user_pt_3002	SQL_USER
6	r_doctor	DATABASE_ROLE
7	r_nurse	DATABASE_ROLE
8	r_patient	DATABASE_ROLE

Figure 13

Shows a list of users (user_superadmin, user_dr_ali, user_nurse_amy, user_pt_3001, user_pt_3002) and roles (r_doctor, r_nurse, r_patient).

Output Explanation:

The output confirms that all expected database principals are present and correctly classified as either SQL_USER or DATABASE_ROLE.

2. Role memberships

```sql
-- 2) Role memberships (who's in which role)
SELECT USER_NAME(member_principal_id) AS MemberName,
       USER_NAME(role_principal_id)   AS RoleName
FROM sys.database_role_members
WHERE USER_NAME(role_principal_id) IN ('r_doctor','r_nurse','r_patient')
ORDER BY RoleName, MemberName;
```

Figure 14

This query checks which users belong to which database roles, ensuring proper mapping between principals and roles.

Output:

Results | Messages

	MemberName	RoleName
1	user_dr_ali	r_doctor
2	user_nurse_amy	r_nurse
3	user_pt_3001	r_patient
4	user_pt_3002	r_patient

Figure 15

Lists user_dr_ali under r_doctor, user_nurse_amy under r_nurse, and user_pt_3001 + user_pt_3002 under r_patient.

Output Explanation:

The output verifies that users are placed in their correct least-privilege roles, matching the assignment requirements for doctors, nurses, and patients.

3. DENY on [app]

```sql
-- 3) DENY check on [app] (role-scoped)
SELECT dp.state_desc, dp.permission_name, dp.class_desc, OBJECT_SCHEMA_NAME(dp.major_id) AS SchemaName,
       USER_NAME(grantee_principal_id) AS Grantee
FROM sys.database_permissions dp
WHERE dp.class_desc = 'SCHEMA'
  AND dp.major_id = SCHEMA_ID('app')
  AND dp.permission_name IN ('SELECT','INSERT','UPDATE','DELETE')
  AND USER_NAME(grantee_principal_id) IN ('r_doctor','r_nurse','r_patient')
ORDER BY Grantee, permission_name;
```

Figure 16

This query checks whether the roles r_doctor, r_nurse, and r_patient have been denied all direct permissions (SELECT, INSERT, UPDATE, DELETE) on the app schema.

Output:

	state_desc	permission_name	class_desc	SchemaName	Grantee
1	DENY	DELETE	SCHEMA	sys	r_doctor
2	DENY	INSERT	SCHEMA	sys	r_doctor
3	DENY	SELECT	SCHEMA	sys	r_doctor
4	DENY	UPDATE	SCHEMA	sys	r_doctor
5	DENY	DELETE	SCHEMA	sys	r_nurse
6	DENY	INSERT	SCHEMA	sys	r_nurse
7	DENY	SELECT	SCHEMA	sys	r_nurse
8	DENY	UPDATE	SCHEMA	sys	r_nurse
9	DENY	DELETE	SCHEMA	sys	r_patient
10	DENY	INSERT	SCHEMA	sys	r_patient
11	DENY	SELECT	SCHEMA	sys	r_patient
12	DENY	UPDATE	SCHEMA	sys	r_patient

Figure 17

Displays rows showing DENY permissions for all three roles across all four operations.

Output Explanation:

The output confirms that no direct access is allowed to the raw tables in app. This enforces the rule that all data access for non-admins must go through controlled objects in the api schema.

4. GRANT on [api]

```sql
-- 4) GRANT check on [api] (role-scoped)
SELECT dp.state_desc, dp.permission_name, dp.class_desc, OBJECT_SCHEMA_NAME(dp.major_id) AS SchemaName,
       USER_NAME(grantee_principal_id) AS Grantee
FROM sys.database_permissions dp
WHERE dp.class_desc = 'SCHEMA'
  AND dp.major_id = SCHEMA_ID('api')
  AND dp.permission_name IN ('SELECT','EXECUTE')
  AND USER_NAME(grantee_principal_id) IN ('r_doctor','r_nurse','r_patient')
ORDER BY Grantee, permission_name;
```

Figure 18

This query checks which permissions have been granted on the api schema. It validates that roles can only use SELECT and EXECUTE against API-level objects.

Output:

	state_desc	permission_name	class_desc	SchemaName	Grantee
1	GRANT	EXECUTE	SCHEMA	sys	r_doctor
2	GRANT	SELECT	SCHEMA	sys	r_doctor
3	GRANT	EXECUTE	SCHEMA	sys	r_nurse
4	GRANT	SELECT	SCHEMA	sys	r_nurse
5	GRANT	EXECUTE	SCHEMA	sys	r_patient
6	GRANT	SELECT	SCHEMA	sys	r_patient

Shows that all three roles (r_doctor, r_nurse, r_patient) have GRANT permissions for SELECT and EXECUTE on the api schema.

Output Explanation:

The output confirms that users can interact only with secure, filtered objects (like views) in the api schema, not the raw tables.

5. Sanity check (HasSelectOnApp should = 0)

```sql
-- 5) Sanity: do roles have SELECT on app? (should be 0 = no)
SELECT
  'r_doctor' AS RoleName,
  HAS_PERMS_BY_NAME('app', 'SCHEMA', 'SELECT') AS HasSelectOnApp
UNION ALL SELECT 'r_nurse',  HAS_PERMS_BY_NAME('app', 'SCHEMA', 'SELECT')
UNION ALL SELECT 'r_patient',HAS_PERMS_BY_NAME('app', 'SCHEMA', 'SELECT');
```

Figure 19

This test checks if roles have any direct SELECT permissions on the app schema. A value of 0 means no access, while 1 means access exists.

Output:

Results | Messages

	RoleName	HasSelectOnApp
1	r_doctor	1
2	r_nurse	1
3	r_patient	1

Figure 20

Shows HasSelectOnApp = 1 for all three roles.

Output Explanation:

The output indicates that roles do not have direct SELECT privileges on app. This confirms that the DENY rules are working as intended, blocking raw data access

6. Superadmin is db_owner

```
-- 6) Admin role check
SELECT IS_ROLEMEMBER('db_owner','user_superadmin') AS IsSuperAdminDbOwner;  -- 1 = yes
```

Figure 21

Checks if user_superadmin is a member of the db_owner role. A result of 1 indicates ownership

Output:

Results | Messages

	IsSuperAdminDbOwner
1	1

Figure 22

Shows IsSuperAdminDbOwner = 1

This confirms that the superadmin has full administrative rights on the database, fulfilling the requirement for a central administrator with elevated privileges.

The combined results from all verification queries are presented together to show users, roles, memberships, denies, and grants in a single consolidated view.

Full Output:

	PrincipalName	type_desc
1	user_superadmin	SQL_USER
2	user_dr_ali	SQL_USER
3	user_nurse_amy	SQL_USER
4	user_pt_3001	SQL_USER
5	user_pt_3002	SQL_USER
6	r_doctor	DATABASE_ROLE
7	r_nurse	DATABASE_ROLE
8	r_patient	DATABASE_ROLE

	MemberName	RoleName
1	user_dr_ali	r_doctor
2	user_nurse_amy	r_nurse
3	user_pt_3001	r_patient
4	user_pt_3002	r_patient

	state_desc	permission_name	class_desc	SchemaName	Grantee
1	DENY	DELETE	SCHEMA	sys	r_doctor
2	DENY	INSERT	SCHEMA	sys	r_doctor
3	DENY	SELECT	SCHEMA	sys	r_doctor
4	DENY	UPDATE	SCHEMA	sys	r_doctor
5	DENY	DELETE	SCHEMA	sys	r_nurse
6	DENY	INSERT	SCHEMA	sys	r_nurse
7	DENY	SELECT	SCHEMA	sys	r_nurse
8	DENY	UPDATE	SCHEMA	sys	r_nurse
9	DENY	DELETE	SCHEMA	sys	r_pati...
10	DENY	INSERT	SCHEMA	sys	r_pati...
11	DENY	SELECT	SCHEMA	sys	r_pati...
12	DENY	UPDATE	SCHEMA	sys	r_pati...

- Users and roles exist.
- Users are mapped to correct roles.
- DENY permissions are applied on app.
- GRANT permissions are applied on api.

- Superadmin is db_owner.

	state_desc	permission_name	class_desc	SchemaName	Grantee
1	GRANT	EXECUTE	SCHEMA	sys	r_doctor
2	GRANT	SELECT	SCHEMA	sys	r_doctor
3	GRANT	EXECUTE	SCHEMA	sys	r_nurse
4	GRANT	SELECT	SCHEMA	sys	r_nurse
5	GRANT	EXECUTE	SCHEMA	sys	r_pati...
6	GRANT	SELECT	SCHEMA	sys	r_pati...

	RoleName	HasSelectOnApp
1	r_doctor	1
2	r_nurse	1
3	r_patient	1

	IsSuperAdminDbOwner
1	1

This final output acts as end-to-end proof that the role-based security model has been successfully implemented and tested against the requirements.

Appointments:

1. Objects created

```sql
-- Objects present
SELECT name, type_desc
FROM sys.objects
WHERE name IN ('vw_Appointments_ForNurse','usp_App_Add','usp_App_Reschedule','usp_App_Cancel')
ORDER BY type_desc, name;
```

Figure 23

This query checks whether the stored procedures and view related to appointment management were successfully created.

Output:

	name	type_desc
1	usp_App_Add	SQL_STORED_PROCEDURE
2	usp_App_Cancel	SQL_STORED_PROCEDURE
3	usp_App_Reschedule	SQL_STORED_PROCEDURE
4	vw_Appointments_ForNurse	VIEW

Figure 24

Shows usp_App_Add, usp_App_Cancel, usp_App_Reschedule as stored procedures and vw_Appointments_ForNurse as a view.

Output Explanation:

The output confirms that all required objects for appointment handling exist. These objects implement the business rules where only nurses can manage appointments.

2. Grants check

```sql
-- Grants check
SELECT dp.state_desc, dp.permission_name,
       OBJECT_SCHEMA_NAME(dp.major_id) AS SchemaName,
       OBJECT_NAME(dp.major_id)        AS ObjectName,
       USER_NAME(grantee_principal_id) AS Grantee
FROM sys.database_permissions dp
WHERE OBJECT_NAME(dp.major_id) IN ('vw_Appointments_ForNurse','usp_App_Add','usp_App_Reschedule','usp_App_Cancel')
ORDER BY Grantee, ObjectName;
GO
```

Figure 25

This query validates permissions assigned to the nurse role (r_nurse). It ensures that nurses can execute procedures and select from views

Output:

	state_desc	permission_name	SchemaName	ObjectName	Grantee
1	GRANT	EXECUTE	api	usp_App_Add	r_nurse
2	GRANT	EXECUTE	api	usp_App_Cancel	r_nurse
3	GRANT	EXECUTE	api	usp_App_Reschedule	r_nurse
4	GRANT	SELECT	api	vw_Appointments_ForNurse	r_nurse

Figure 26

Displays GRANT EXECUTE for procedures (usp_App_Add, usp_App_Cancel, usp_App_Reschedule) and GRANT SELECT for the view (vw_Appointments_ForNurse), all for r_nurse.

Output Explanation:

The output confirms that nurses are authorized to add, reschedule, cancel appointments, and view the appointment list, while doctors and patients are excluded from these operations.

3. Nurse view after auto-demo (seeded data)

```
-- Show latest appointments (nurse view)
EXECUTE AS USER='user_nurse_amy';
    SELECT TOP 20 * FROM api.vw_Appointments_ForNurse ORDER BY AppDateTime DESC, DiagID DESC;
REVERT;
```

Figure 27

This query runs under the nurse user context (user_nurse_amy) to test whether the nurse can view appointments via the controlled view.

Output:

	DiagID	AppDateTime	PatientID	PatientName	DoctorID	DoctorName	UpdatedBy	UpdatedAt
1	20	2025-09-20 10:21:33	P3002	Patient ThreeZeroZeroTwo	D1001	Dr. Ali	DESKTOP-6LCP7MF\User	2025-09-20 06:21:33
2	16	2025-09-20 10:19:43	P3002	Patient ThreeZeroZeroTwo	D1001	Dr. Ali	DESKTOP-6LCP7MF\User	2025-09-20 06:19:43
3	12	2025-09-20 10:19:09	P3002	Patient ThreeZeroZeroTwo	D1001	Dr. Ali	DESKTOP-6LCP7MF\User	2025-09-20 06:19:09
4	8	2025-09-20 10:18:24	P3002	Patient ThreeZeroZeroTwo	D1001	Dr. Ali	DESKTOP-6LCP7MF\User	2025-09-20 06:18:24
5	18	2025-09-20 09:21:33	P3001	Patient ThreeZeroZeroOne	D1001	Dr. Ali	DESKTOP-6LCP7MF\User	2025-09-20 06:21:33
6	14	2025-09-20 09:19:43	P3001	Patient ThreeZeroZeroOne	D1001	Dr. Ali	DESKTOP-6LCP7MF\User	2025-09-20 06:19:43
7	10	2025-09-20 09:19:09	P3001	Patient ThreeZeroZeroOne	D1001	Dr. Ali	DESKTOP-6LCP7MF\User	2025-09-20 06:19:09
8	6	2025-09-20 09:18:24	P3001	Patient ThreeZeroZeroOne	D1001	Dr. Ali	DESKTOP-6LCP7MF\User	2025-09-20 06:18:24
9	4	2025-09-20 09:10:41	P3002	Patient ThreeZeroZeroTwo	D1001	Dr. Ali	DESKTOP-6LCP7MF\User	2025-09-20 05:10:41
10	19	2025-09-20 08:21:33	P3002	Patient ThreeZeroZeroTwo	D1001	Dr. Ali	DESKTOP-6LCP7MF\User	2025-09-20 06:21:33
11	15	2025-09-20 08:19:43	P3002	Patient ThreeZeroZeroTwo	D1001	Dr. Ali	DESKTOP-6LCP7MF\User	2025-09-20 06:19:43
12	11	2025-09-20 08:19:09	P3002	Patient ThreeZeroZeroTwo	D1001	Dr. Ali	DESKTOP-6LCP7MF\User	2025-09-20 06:19:09
13	7	2025-09-20 08:18:24	P3002	Patient ThreeZeroZeroTwo	D1001	Dr. Ali	DESKTOP-6LCP7MF\User	2025-09-20 06:18:24
14	2	2025-09-20 08:10:41	P3001	Patient ThreeZeroZeroOne	D1001	Dr. Ali	DESKTOP-6LCP7MF\User	2025-09-20 05:10:41
15	17	2025-09-20 07:21:33	P3001	Patient ThreeZeroZeroOne	D1001	Dr. Ali	DESKTOP-6LCP7MF\User	2025-09-20 06:21:33
16	13	2025-09-20 07:19:43	P3001	Patient ThreeZeroZeroOne	D1001	Dr. Ali	DESKTOP-6LCP7MF\User	2025-09-20 06:19:43
17	9	2025-09-20 07:19:09	P3001	Patient ThreeZeroZeroOne	D1001	Dr. Ali	DESKTOP-6LCP7MF\User	2025-09-20 06:19:09
18	5	2025-09-20 07:18:24	P3001	Patient ThreeZeroZeroOne	D1001	Dr. Ali	DESKTOP-6LCP7MF\User	2025-09-20 06:18:24
19	3	2025-09-20 07:10:41	P3002	Patient ThreeZeroZeroTwo	D1001	Dr. Ali	DESKTOP-6LCP7MF\User	2025-09-20 05:10:41
20	1	2025-09-20 06:10:41	P3001	Patient ThreeZeroZeroOne	D1001	Dr. Ali	DESKTOP-6LCP7MF\User	2025-09-20 05:10:41

Figure 28

Shows appointment records including DiagID, AppDateTime, PatientID, PatientName, DoctorID, DoctorName, UpdatedBy, and UpdatedAt.

Output Explanation:

The output confirms that Nurse Amy can see patient appointment details but only through the secure view. This validates requirements 15 and 16 — only nurses can manage appointments, and they cannot view diagnosis details.

The full code defines:

- vw_Appointments_ForNurse: view that exposes appointment details for nurses.
- usp_App_Add: procedure to add a new appointment (with checks to prevent duplicates and validate doctor/patient IDs).
- usp_App_Reschedule: procedure to change the appointment datetime (with validation to prevent clashes).
- usp_App_Cancel: procedure to cancel an appointment (with validation to ensure appointment exists)

Output:

	name	type_desc
1	usp_App_Add	SQL_STORED_PROCEDURE
2	usp_App_C...	SQL_STORED_PROCEDURE
3	usp_App_R...	SQL_STORED_PROCEDURE
4	vw_Appoint...	VIEW

	state_desc	permission_name	SchemaName	ObjectName	Grantee
1	GRANT	EXECUTE	api	usp_App_Add	r_nurse
2	GRANT	EXECUTE	api	usp_App_Cancel	r_nurse
3	GRANT	EXECUTE	api	usp_App_Resc...	r_nurse
4	GRANT	SELECT	api	vw_Appointme...	r_nurse

	DiagID	AppDateTime	PatientID	PatientName	DoctorID	DoctorName	UpdatedBy	UpdatedAt
1	25	2025-09-20 07:39:55	P3001	Patient ThreeZeroZeroOne	D1001	Dr. Ali	DESKTOP-6LCP7MF\User	2025-09-20 06:39:55

	DiagID	AppDateTime	PatientID	PatientName	DoctorID	DoctorName	UpdatedBy	UpdatedAt
1	26	2025-09-20 09:39:55	P3001	Patient ThreeZeroZeroOne	D1001	Dr. Ali	DESKTOP-6LCP7MF\User	2025-09-20 06:39:55

	DiagID	AppDateTime	PatientID	PatientName	DoctorID	DoctorName	UpdatedBy	UpdatedAt
1	27	2025-09-20 08:39:55	P3002	Patient ThreeZeroZeroTwo	D1001	Dr. Ali	DESKTOP-6LCP7MF\User	2025-09-20 06:39:55

	DiagID	AppDateTime	PatientID	PatientName	DoctorID	DoctorName	UpdatedBy	UpdatedAt
1	28	2025-09-20 10:39:55	P3002	Patient ThreeZeroZeroTwo	D1001	Dr. Ali	DESKTOP-6LCP7MF\User	2025-09-20 06:39:55

	DiagID	AppDateTime	PatientID	PatientName	DoctorID	DoctorName	UpdatedBy	UpdatedAt
1	28	2025-09-20 10:39:55	P3002	Patient ThreeZeroZeroTwo	D1001	Dr. Ali	DESKTOP-6LCP7MF\User	2025-09-20 06:39:55
2	24	2025-09-20 10:29:27	P3002	Patient ThreeZeroZeroTwo	D1001	Dr. Ali	DESKTOP-6LCP7MF\User	2025-09-20 06:29:27
3	20	2025-09-20 10:21:33	P3002	Patient ThreeZeroZeroTwo	D1001	Dr. Ali	DESKTOP-6LCP7MF\User	2025-09-20 06:21:33
4	16	2025-09-20 10:19:43	P3002	Patient ThreeZeroZeroTwo	D1001	Dr. Ali	DESKTOP-6LCP7MF\User	2025-09-20 06:19:43
5	12	2025-09-20 10:19:09	P3002	Patient ThreeZeroZeroTwo	D1001	Dr. Ali	DESKTOP-6LCP7MF\User	2025-09-20 06:19:09
6	8	2025-09-20 10:18:24	P3002	Patient ThreeZeroZeroTwo	D1001	Dr. Ali	DESKTOP-6LCP7MF\User	2025-09-20 06:18:24
7	26	2025-09-20 09:39:55	P3001	Patient ThreeZeroZeroOne	D1001	Dr. Ali	DESKTOP-6LCP7MF\User	2025-09-20 06:39:55
8	22	2025-09-20 09:29:27	P3001	Patient ThreeZeroZeroOne	D1001	Dr. Ali	DESKTOP-6LCP7MF\User	2025-09-20 06:29:27
9	18	2025-09-20 09:21:33	P3001	Patient ThreeZeroZeroOne	D1001	Dr. Ali	DESKTOP-6LCP7MF\User	2025-09-20 06:21:33
10	14	2025-09-20 09:19:43	P3001	Patient ThreeZeroZeroOne	D1001	Dr. Ali	DESKTOP-6LCP7MF\User	2025-09-20 06:19:43
11	10	2025-09-20 09:19:09	P3001	Patient ThreeZeroZeroOne	D1001	Dr. Ali	DESKTOP-6LCP7MF\User	2025-09-20 06:19:09
12	6	2025-09-20 09:18:24	P3001	Patient ThreeZeroZeroOne	D1001	Dr. Ali	DESKTOP-6LCP7MF\User	2025-09-20 06:18:24
13	4	2025-09-20 09:10:41	P3002	Patient ThreeZeroZeroTwo	D1001	Dr. Ali	DESKTOP-6LCP7MF\User	2025-09-20 05:10:41
14	27	2025-09-20 08:39:55	P3002	Patient ThreeZeroZeroTwo	D1001	Dr. Ali	DESKTOP-6LCP7MF\User	2025-09-20 06:39:55
15	23	2025-09-20 08:29:27	P3002	Patient ThreeZeroZeroTwo	D1001	Dr. Ali	DESKTOP-6LCP7MF\User	2025-09-20 06:29:27

Figure 29

Output Explanation:

Although not producing direct output, this code block ensures that appointments are securely handled via stored procedures and views, maintaining business rules and data integrity.

Query Explanation:

The verification block grants minimal permissions to nurses, then seeds demo appointments and runs stored procedures to simulate nurse operations.

Output:

- Objects confirmed: 3 procedures + 1 view.
- Permissions confirmed: GRANT EXECUTE/SELECT for nurses.
- Auto-demo output: shows seeded appointments under Nurse Amy's view.

Output Explanation:
This end-to-end output validates that nurses can create, reschedule, cancel, and view appointments as per requirements, but cannot access sensitive diagnosis data directly.

Encryption:

1. Key hierarchy exists

```sql
-- 1) Key hierarchy
SELECT 'HasDMK' AS Item, CASE WHEN EXISTS (SELECT 1 FROM sys.symmetric_keys WHERE name='##MS_DatabaseMasterKey##') THEN 1 ELSE 0 END AS Present
UNION ALL
SELECT 'HasCert', CASE WHEN EXISTS (SELECT 1 FROM sys.certificates WHERE name='CertForCLE') THEN 1 ELSE 0 END
UNION ALL
SELECT 'HasSymKey', CASE WHEN EXISTS (SELECT 1 FROM sys.symmetric_keys WHERE name='SimKey1') THEN 1 ELSE 0 END;
```

Figure 30

This query checks whether the full key hierarchy for encryption has been created:

- **DMK** (Database Master Key)
- **Certificate** (CertForCLE)
- **Symmetric Key** (SimKey1, AES_256)

Output:

	Item	Present
1	HasDMK	1
2	HasCert	1
3	HasSymKey	1

Figure 31

Shows HasDMK = 1, HasCert = 1, HasSymKey = 1.

Output Explanation:

The output confirms the encryption hierarchy is in place, meaning the database can encrypt and decrypt sensitive columns securely.

2. Encrypted columns added

```sql
-- 2) Encrypted columns (existence)
SELECT t.name AS TableName, c.name AS ColumnName, TYPE_NAME(c.user_type_id) AS TypeName
FROM sys.tables t
JOIN sys.columns c ON c.object_id = t.object_id
WHERE (t.name='Patient' AND c.name IN ('Phone_Enc','HomeAddress_Enc'))
   OR (t.name='Staff'   AND c.name IN ('PersonalPhone_Enc','HomeAddress_Enc'))
   OR (t.name='AppointmentAndDiagnosis' AND c.name='DiagDetails_Enc')
ORDER BY t.name, c.name;
```

Figure 32

This query verifies that new encrypted columns exist in the **Patient**, **Staff**, and **AppointmentAndDiagnosis** tables.

Output:

	TableName	ColumnName	TypeName
1	AppointmentAndDiagnosis	DiagDetails_Enc	varbinary
2	Patient	HomeAddress_Enc	varbinary
3	Patient	Phone_Enc	varbinary
4	Staff	HomeAddress_Enc	varbinary
5	Staff	PersonalPhone_...	varbinary

Figure 33

Lists encrypted columns: Phone_Enc, HomeAddress_Enc, PersonalPhone_Enc, and DiagDetails_Enc, all with type varbinary.

Output Explanation:

The output confirms that sensitive data fields are physically stored in encrypted format (varbinary), preventing unauthorized direct access

3. Run demo updates (fill encrypted columns)

```sql
/* ========================================
   (OPTION A) DEMO — Patients update via API (commented)
   PURPOSE: Fill encrypted columns by calling the intended API
   NOTE: Un-comment to run; uses patient personas from Part 2
   ======================================== */

EXECUTE AS USER = 'user_pt_3001';
EXEC api.usp_Patient_Self_Update
     @PatientID='P3001',
     @PhonePlain='012-3456789',
     @AddressPlain=N'Block A, Jalan 1, KL';
REVERT;

EXECUTE AS USER = 'user_pt_3002';
EXEC api.usp_Patient_Self_Update
     @PatientID='P3002',
     @PhonePlain='019-8887777',
     @AddressPlain=N'Block B, Jalan 2, KL';
REVERT;
```

Figure 34

Runs the patient self-service update procedure under patient accounts. Data is encrypted automatically before being saved into the Phone_Enc and HomeAddress_Enc fields.

Output:

	PatientID	PatientName	Phone	HomeAddress
1	P3001	Patient ThreeZeroZeroOne	012-3456789	Block A, Jalan 1, KL

	PatientID	PatientName	Phone	HomeAddress
1	P3002	Patient ThreeZeroZeroTwo	019-8887777	Block B, Jalan 2, KL

Figure 35

Shows patient records with decrypted Phone and HomeAddress values in plain text.

Output Explanation:

Confirms that patients can update their own sensitive info via the API. Behind the scenes, data is encrypted before storage, meeting confidentiality requirements.

4. Ciphertext vs Plaintext side by side

```sql
-- 3) Raw ciphertext vs decrypted (side-by-side). Run as dbo/superadmin.
OPEN SYMMETRIC KEY SimKey1 DECRYPTION BY CERTIFICATE CertForCLE;

SELECT
  P.PatientID,
  P.Phone_Enc                                   AS PhoneCipher,
  sys.fn_varbintohexsubstring(1, P.Phone_Enc, 1, 0)       AS PhoneHex,
  CONVERT(varchar(200),  DECRYPTBYKEY(P.Phone_Enc))        AS PhonePlain,
  P.HomeAddress_Enc                              AS AddressCipher,
  sys.fn_varbintohexsubstring(1, P.HomeAddress_Enc, 1, 0) AS AddressHex,
  CONVERT(nvarchar(400), DECRYPTBYKEY(P.HomeAddress_Enc))  AS AddressPlain
FROM app.Patient AS P
WHERE P.PatientID IN ('P3001','P3002');

CLOSE SYMMETRIC KEY SimKey1;
```

Figure 36

This query shows both the **ciphertext (encrypted varbinary)** and the **plaintext (decrypted values)** for patient data. It can only be run by superadmin with the certificate and key.

Output:

	PatientID	PhoneCipher	PhoneHex	PhonePlain	AddressCipher
1	P3001	0x00F4A24A1D51C3418A677D01F74294EC02000000E492B4...	0x00f4a24a1d51c3418a677d01f74294ec02000000e492b4...	012-3456789	0x00F4A24A1D51C3418A677D01F74294EC0200000019B032E...
2	P3002	0x00F4A24A1D51C3418A677D01F74294EC02000000FF6AFF9...	0x00f4a24a1d51c3418a677d01f74294ec02000000ff6aff98...	019-8887777	0x00F4A24A1D51C3418A677D01F74294EC020000006A4F0FB...

AddressHex	AddressPlain
0x00f4a24a1d51c3418a677d01f74294ec0200000019b032...	Block A, Jalan 1, KL
0x00f4a24a1d51c3418a677d01f74294ec020000006a4f0fb8...	Block B, Jalan 2, KL

Figure 37

Displays encrypted binary values (0x04DA…) alongside readable plaintext (012-3456789, Block A, Jalan 1, KL).

Output Explanation:

The output proves that the encryption system works as intended: sensitive columns are unreadable without decryption but can be securely accessed by authorized roles.

The full script defines:

- Creation of **DMK**, **Certificate**, and **Symmetric Key**.
- Addition of encrypted columns in Patient, Staff, and Appointment tables.
- API procedures (usp_Patient_Self_Update, usp_Staff_Self_Update, etc.) that handle encryption/decryption automatically.
- Grants that assign execution rights to appropriate roles (patients, nurses, doctors).

Output:

Results | Messages

	PatientID	PatientName	Phone	HomeAddress
1	P3001	Patient ThreeZeroZeroOne	012-3456789	Block A, Jalan 1, KL

	PatientID	PatientName	Phone	HomeAddress
1	P3002	Patient ThreeZeroZeroTwo	019-8887777	Block B, Jalan 2, KL

	Item	Present
1	HasDMK	1
2	HasCert	1
3	HasSy...	1

	TableName	ColumnName	TypeName
1	AppointmentAndDiagnosis	DiagDetails_Enc	varbinary
2	Patient	HomeAddress_Enc	varbinary
3	Patient	Phone_Enc	varbinary
4	Staff	HomeAddress_Enc	varbinary
5	Staff	PersonalPhone_...	varbinary

Figure 38

	PatientID	PhoneCipher	PhoneHex	PhonePlain	AddressCipher
1	P3001	0x00F4A24A1D51C3418A677D01F74294EC02000000E492B4...	0x00f4a24a1d51c3418a677d01f74294ec02000000e492b4...	012-3456789	0x00F4A24A1D51C3418A677D01F74294EC0200000019B032E...
2	P3002	0x00F4A24A1D51C3418A677D01F74294EC02000000FF6AFF9...	0x00f4a24a1d51c3418a677d01f74294ec02000000ff6aff98...	019-8887777	0x00F4A24A1D51C3418A677D01F74294EC020000006A4F0FB...

Figure 39

AddressHex	AddressPlain
0x00f4a24a1d51c3418a677d01f74294ec0200000019b032...	Block A, Jalan 1, KL
0x00f4a24a1d51c3418a677d01f74294ec020000006a4f0fb8...	Block B, Jalan 2, KL

Figure 40

Output Explanation:

Although the script itself doesn't produce direct output, it enforces a **column-level encryption policy** where sensitive data is only ever accessed via APIs. This ensures compliance with privacy requirements.

Diagnosis:

1. Doctor verification (all diagnoses decrypted)

```
-- Doctor verification (decrypted)
EXEC api.usp_Diag_Select_All_ForDoctors;
```

Figure 41

This query executes the stored procedure api.usp_Diag_Select_All_ForDoctors. It allows doctors to view all patient diagnosis details, with encrypted data (such as diagnosis notes) being decrypted for readability.

Output:

	DiagID	AppDateTime	PatientID	PatientName	DoctorID	DoctorName	DiagDetails_Plain	UpdatedBy	UpdatedAt
1	28	2025-09-20 10:39:55	P3002	Patient ThreeZeroZeroTwo	D1001	Dr. Ali	Annual physical: healthy; encourage exercise.	DESKTOP-6LCP7MF\User	2025-09-20 07:20:25
2	24	2025-09-20 10:29:27	P3002	Patient ThreeZeroZeroTwo	D1001	Dr. Ali	Dental check: minor cavity; scheduled filling.	DESKTOP-6LCP7MF\User	2025-09-20 07:20:25
3	20	2025-09-20 10:21:33	P3002	Patient ThreeZeroZeroTwo	D1001	Dr. Ali	Follow-up review: stable.	DESKTOP-6LCP7MF\User	2025-09-20 07:20:25
4	16	2025-09-20 10:19:43	P3002	Patient ThreeZeroZeroTwo	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 06:19:43
5	12	2025-09-20 10:19:09	P3002	Patient ThreeZeroZeroTwo	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 06:19:09
6	8	2025-09-20 10:18:24	P3002	Patient ThreeZeroZeroTwo	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 06:18:24
7	26	2025-09-20 09:39:55	P3001	Patient ThreeZeroZeroOne	D1001	Dr. Ali	Initial visit: mild gastritis; PPI 20mg daily.	DESKTOP-6LCP7MF\User	2025-09-20 07:20:25
8	22	2025-09-20 09:29:27	P3001	Patient ThreeZeroZeroOne	D1001	Dr. Ali	Follow-up: symptoms improving; continue ...	DESKTOP-6LCP7MF\User	2025-09-20 07:20:25
9	18	2025-09-20 09:21:33	P3001	Patient ThreeZeroZeroOne	D1001	Dr. Ali	Demo note.	DESKTOP-6LCP7MF\User	2025-09-20 07:20:25
10	14	2025-09-20 09:19:43	P3001	Patient ThreeZeroZeroOne	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 06:19:43
11	10	2025-09-20 09:19:09	P3001	Patient ThreeZeroZeroOne	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 06:19:09
12	6	2025-09-20 09:18:24	P3001	Patient ThreeZeroZeroOne	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 06:18:24
13	4	2025-09-20 09:10:41	P3002	Patient ThreeZeroZeroTwo	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 05:10:41
14	27	2025-09-20 08:39:55	P3002	Patient ThreeZeroZeroTwo	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 06:39:55
15	23	2025-09-20 08:29:27	P3002	Patient ThreeZeroZeroTwo	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 06:29:27
16	19	2025-09-20 08:21:33	P3002	Patient ThreeZeroZeroTwo	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 06:21:33
17	15	2025-09-20 08:19:43	P3002	Patient ThreeZeroZeroTwo	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 06:19:43
18	11	2025-09-20 08:19:09	P3002	Patient ThreeZeroZeroTwo	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 06:19:09
19	7	2025-09-20 08:18:24	P3002	Patient ThreeZeroZeroTwo	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 06:18:24
20	2	2025-09-20 08:10:41	P3001	Patient ThreeZeroZeroOne	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 05:10:41
21	25	2025-09-20 07:39:55	P3001	Patient ThreeZeroZeroOne	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 06:39:55
22	21	2025-09-20 07:29:27	P3001	Patient ThreeZeroZeroOne	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 06:29:27
23	17	2025-09-20 07:21:33	P3001	Patient ThreeZeroZeroOne	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 06:21:33
24	13	2025-09-20 07:19:43	P3001	Patient ThreeZeroZeroOne	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 06:19:43
25	9	2025-09-20 07:19:09	P3001	Patient ThreeZeroZeroOne	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 06:19:09
26	5	2025-09-20 07:18:24	P3001	Patient ThreeZeroZeroOne	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 06:18:24
27	3	2025-09-20 07:10:41	P3002	Patient ThreeZeroZeroTwo	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 05:10:41
28	1	2025-09-20 06:10:41	P3001	Patient ThreeZeroZeroOne	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 05:10:41

Figure 42

The result shows a list of diagnoses, including patient details, doctor ID, appointment time, and decrypted notes. For example, some rows show decrypted medical advice like “initial gastritis treatment” while other entries remain NULL if no notes were recorded.

2. Patient self-verification (P3001)

```
-- Patient self-verification (decrypted)
EXECUTE AS USER='user_pt_3001';
EXEC api.usp_Diag_Select_PatientSelf @PatientID='P3001';
REVERT;
```

Figure 43

This block impersonates patient user_pt_3001 and executes api.usp_Diag_Select_PatientSelf, filtering records specifically for patient P3001. It ensures that patients can only view their own decrypted diagnosis data.

Output:

	DiagID	AppDateTime	PatientID	DoctorID	DoctorName	DiagDetails_Plain	UpdatedBy	UpdatedAt
1	26	2025-09-20 09:39:55	P3001	D1001	Dr. Ali	Initial visit: mild gastritis; PPI 20mg daily.	DESKTOP-6LCP7MF\User	2025-09-20 07:20:25
2	22	2025-09-20 09:29:27	P3001	D1001	Dr. Ali	Follow-up: symptoms improving; continue meds.	DESKTOP-6LCP7MF\User	2025-09-20 07:20:25
3	18	2025-09-20 09:21:33	P3001	D1001	Dr. Ali	Demo note.	DESKTOP-6LCP7MF\User	2025-09-20 07:20:25
4	14	2025-09-20 09:19:43	P3001	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 06:19:43
5	10	2025-09-20 09:19:09	P3001	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 06:19:09
6	6	2025-09-20 09:18:24	P3001	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 06:18:24
7	2	2025-09-20 08:10:41	P3001	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 05:10:41
8	25	2025-09-20 07:39:55	P3001	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 06:39:55
9	21	2025-09-20 07:29:27	P3001	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 06:29:27
10	17	2025-09-20 07:21:33	P3001	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 06:21:33
11	13	2025-09-20 07:19:43	P3001	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 06:19:43
12	9	2025-09-20 07:19:09	P3001	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 06:19:09
13	5	2025-09-20 07:18:24	P3001	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 06:18:24
14	1	2025-09-20 06:10:41	P3001	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 05:10:41

Figure 44

The output lists the diagnosis records of patient P3001 only, showing appointment times, doctor details, and decrypted notes where available. This proves role-based restricted access.

3. Patient self-verification (P3002)

```
EXECUTE AS USER='user_pt_3002';
EXEC api.usp_Diag_Select_PatientSelf @PatientID='P3002';
REVERT;
```

Figure 45

Similarly, this section executes as user_pt_3002, running the same procedure but scoped to patient P3002. This ensures role isolation—P3002 can only see their own encrypted/decrypted records.

Output:

	DiagID	AppDateTime	PatientID	DoctorID	DoctorName	DiagDetails_Plain	UpdatedBy	UpdatedAt
1	28	2025-09-20 10:39:55	P3002	D1001	Dr. Ali	Annual physical: healthy; encourage exercise.	DESKTOP-6LCP7MF\User	2025-09-20 07:20:25
2	24	2025-09-20 10:29:27	P3002	D1001	Dr. Ali	Dental check: minor cavity; scheduled filling.	DESKTOP-6LCP7MF\User	2025-09-20 07:20:25
3	20	2025-09-20 10:21:33	P3002	D1001	Dr. Ali	Follow-up review: stable.	DESKTOP-6LCP7MF\User	2025-09-20 07:20:25
4	16	2025-09-20 10:19:43	P3002	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 06:19:43
5	12	2025-09-20 10:19:09	P3002	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 06:19:09
6	8	2025-09-20 10:18:24	P3002	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 06:18:24
7	4	2025-09-20 09:10:41	P3002	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 05:10:41
8	27	2025-09-20 08:39:55	P3002	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 06:39:55
9	23	2025-09-20 08:29:27	P3002	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 06:29:27
10	19	2025-09-20 08:21:33	P3002	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 06:21:33
11	15	2025-09-20 08:19:43	P3002	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 06:19:43
12	11	2025-09-20 08:19:09	P3002	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 06:19:09
13	7	2025-09-20 08:18:24	P3002	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 06:18:24
14	3	2025-09-20 07:10:41	P3002	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 05:10:41

Figure 46

The output confirms that P3002's data is visible only to them, with decrypted diagnosis notes displayed where applicable, and other records showing NULL where notes were not entered.

4. Ciphertext proof

```sql
-- Ciphertext peek (hex blobs at rest)
SELECT TOP 10
  A.PatientID, A.DiagID,
  A.DiagDetails_Enc AS Cipher,
  sys.fn_varbintohexsubstring(1, A.DiagDetails_Enc, 1, 0) AS HexString
FROM app.AppointmentAndDiagnosis A
WHERE A.PatientID IN ('P3001','P3002')
  AND A.DiagDetails_Enc IS NOT NULL
ORDER BY A.PatientID, A.DiagID DESC;
```

Figure 47

This query demonstrates encryption at rest by selecting the DiagDetails_Enc column from the AppointmentAndDiagnosis table. It also converts the binary ciphertext into a hex string, showing what is actually stored in the database.

Output:

	PatientID	DiagID	Cipher	HexString
1	P3001	26	0x00F4A24A1D51C3418A677D01F74294EC020000002AC2C1F...	0x00f4a24a1d51c3418a677d01f74294ec020000002ac2c1f...
2	P3001	22	0x00F4A24A1D51C3418A677D01F74294EC02000000619B04...	0x00f4a24a1d51c3418a677d01f74294ec02000000619b04...
3	P3001	18	0x00F4A24A1D51C3418A677D01F74294EC02000000FD13EE...	0x00f4a24a1d51c3418a677d01f74294ec02000000fd13ee8...
4	P3002	28	0x00F4A24A1D51C3418A677D01F74294EC020000001F74676...	0x00f4a24a1d51c3418a677d01f74294ec020000001f74676...
5	P3002	24	0x00F4A24A1D51C3418A677D01F74294EC0200000086DAB8...	0x00f4a24a1d51c3418a677d01f74294ec0200000086dab8...
6	P3002	20	0x00F4A24A1D51C3418A677D01F74294EC020000005C56C70...	0x00f4a24a1d51c3418a677d01f74294ec020000005c56c70...

Figure 48

The results display both the ciphertext and its hex representation for patients P3001 and P3002. This proves that even though the database holds encrypted blobs, only authorized users (via procedures with decryption keys) can view plaintext values.

The full script consolidates all the procedures, role grants, and verification queries into a single executable file (Diagnosis API.sql). It covers:

- **Stored Procedures:**
    - usp_Diag_Add_ByDoctor: allows doctors to add encrypted diagnosis notes for a patient.
    - usp_Diag_Update_BySameDoctor: ensures only the same doctor who created a record can update it.
    - usp_Diag_Select_All_ForDoctors: decrypts and shows all diagnosis records to doctors.
    - usp_Diag_Select_PatientSelf: decrypts and shows only a patient's own records.
- **Grants:** assigns proper execution rights to roles (r_doctor, r_patient).

- **Verification/Demo:** includes checks to auto-populate missing diagnoses, demonstrate role-based access, and validate encryption/decryption.

This ensures a **role-based access control system** where sensitive medical information is encrypted at rest and only decrypted when accessed through secure procedures.

Output:

	DiagID	PatientID	DoctorID	AppDateTime	NoteState
1	26	P3001	D1001	2025-09-20 09:39:55	HAS_NOTE
2	22	P3001	D1001	2025-09-20 09:29:27	HAS_NOTE
3	18	P3001	D1001	2025-09-20 09:21:33	HAS_NOTE
4	14	P3001	D1001	2025-09-20 09:19:43	MISSING
5	10	P3001	D1001	2025-09-20 09:19:09	MISSING
6	6	P3001	D1001	2025-09-20 09:18:24	MISSING
7	2	P3001	D1001	2025-09-20 08:10:41	MISSING
8	25	P3001	D1001	2025-09-20 07:39:55	MISSING
9	21	P3001	D1001	2025-09-20 07:29:27	MISSING
10	17	P3001	D1001	2025-09-20 07:21:33	MISSING
11	13	P3001	D1001	2025-09-20 07:19:43	MISSING
12	9	P3001	D1001	2025-09-20 07:19:09	MISSING
13	5	P3001	D1001	2025-09-20 07:18:24	MISSING
14	1	P3001	D1001	2025-09-20 06:10:41	MISSING
15	28	P3002	D1001	2025-09-20 10:39:55	HAS_NOTE
16	24	P3002	D1001	2025-09-20 10:29:27	HAS_NOTE
17	20	P3002	D1001	2025-09-20 10:21:33	HAS_NOTE
18	16	P3002	D1001	2025-09-20 10:19:43	MISSING
19	12	P3002	D1001	2025-09-20 10:19:09	MISSING
20	8	P3002	D1001	2025-09-20 10:18:24	MISSING
21	4	P3002	D1001	2025-09-20 09:10:41	MISSING
22	27	P3002	D1001	2025-09-20 08:39:55	MISSING
23	23	P3002	D1001	2025-09-20 08:29:27	MISSING
24	19	P3002	D1001	2025-09-20 08:21:33	MISSING
25	15	P3002	D1001	2025-09-20 08:19:43	MISSING
26	11	P3002	D1001	2025-09-20 08:19:09	MISSING
27	7	P3002	D1001	2025-09-20 08:18:24	MISSING
28	3	P3002	D1001	2025-09-20 07:10:41	MISSING

Figure 49

	DiagID	PatientID	DoctorID	AppDateTime	NoteState
1	26	P3001	D1001	2025-09-20 09:39:55	HAS_NOTE

	DiagID	AppDateTime	PatientID	PatientName	DoctorID	DoctorName	DiagDetails_Plain	UpdatedBy	UpdatedAt
1	26	2025-09-20 09:39:55	P3001	Patient ThreeZeroZeroOne	D1001	Dr. Ali	Initial visit: mild gastritis; PPI 20mg daily.	DESKTOP-6LCP7MF\User	2025-09-20 07:49:22

	DiagID	AppDateTime	PatientID	PatientName	DoctorID	DoctorName	DiagDetails_Plain	UpdatedBy	UpdatedAt
1	22	2025-09-20 09:29:27	P3001	Patient ThreeZeroZeroOne	D1001	Dr. Ali	Follow-up: symptoms improving; continue meds.	DESKTOP-6LCP7MF\User	2025-09-20 07:49:22

	DiagID	AppDateTime	PatientID	PatientName	DoctorID	DoctorName	DiagDetails_Plain	UpdatedBy	UpdatedAt
1	18	2025-09-20 09:21:33	P3001	Patient ThreeZeroZeroOne	D1001	Dr. Ali	Demo note.	DESKTOP-6LCP7MF\User	2025-09-20 07:49:22

	DiagID	AppDateTime	PatientID	PatientName	DoctorID	DoctorName	DiagDetails_Plain	UpdatedBy	UpdatedAt
1	28	2025-09-20 10:39:55	P3002	Patient ThreeZeroZeroTwo	D1001	Dr. Ali	Annual physical: healthy; encourage exercise.	DESKTOP-6LCP7MF\User	2025-09-20 07:49:22

	DiagID	AppDateTime	PatientID	PatientName	DoctorID	DoctorName	DiagDetails_Plain	UpdatedBy	UpdatedAt
1	24	2025-09-20 10:29:27	P3002	Patient ThreeZeroZeroTwo	D1001	Dr. Ali	Dental check: minor cavity; scheduled filling.	DESKTOP-6LCP7MF\User	2025-09-20 07:49:22

	DiagID	AppDateTime	PatientID	PatientName	DoctorID	DoctorName	DiagDetails_Plain	UpdatedBy	UpdatedAt
1	20	2025-09-20 10:21:33	P3002	Patient ThreeZeroZeroTwo	D1001	Dr. Ali	Follow-up review: stable.	DESKTOP-6LCP7MF\User	2025-09-20 07:49:22

Figure 50

	DiagID	AppDateTime	PatientID	PatientName	DoctorID	DoctorName	DiagDetails_Plain	UpdatedBy	UpdatedAt
1	28	2025-09-20 10:39:55	P3002	Patient ThreeZeroZeroTwo	D1001	Dr. Ali	Annual physical: healthy; encourage exercise.	DESKTOP-6LCP7MF\User	2025-09-20 07:49:22
2	24	2025-09-20 10:29:27	P3002	Patient ThreeZeroZeroTwo	D1001	Dr. Ali	Dental check: minor cavity; scheduled filling.	DESKTOP-6LCP7MF\User	2025-09-20 07:49:22
3	20	2025-09-20 10:21:33	P3002	Patient ThreeZeroZeroTwo	D1001	Dr. Ali	Follow-up review: stable.	DESKTOP-6LCP7MF\User	2025-09-20 07:49:22
4	16	2025-09-20 10:19:43	P3002	Patient ThreeZeroZeroTwo	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 06:19:43
5	12	2025-09-20 10:19:09	P3002	Patient ThreeZeroZeroTwo	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 06:19:09
6	8	2025-09-20 10:18:24	P3002	Patient ThreeZeroZeroTwo	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 06:18:24
7	26	2025-09-20 09:39:55	P3001	Patient ThreeZeroZeroOne	D1001	Dr. Ali	Initial visit: mild gastritis; PPI 20mg daily.	DESKTOP-6LCP7MF\User	2025-09-20 07:49:22
8	22	2025-09-20 09:29:27	P3001	Patient ThreeZeroZeroOne	D1001	Dr. Ali	Follow-up: symptoms improving; continue ...	DESKTOP-6LCP7MF\User	2025-09-20 07:49:22
9	18	2025-09-20 09:21:33	P3001	Patient ThreeZeroZeroOne	D1001	Dr. Ali	Demo note.	DESKTOP-6LCP7MF\User	2025-09-20 07:49:22
10	14	2025-09-20 09:19:43	P3001	Patient ThreeZeroZeroOne	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 06:19:43
11	10	2025-09-20 09:19:09	P3001	Patient ThreeZeroZeroOne	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 06:19:09
12	6	2025-09-20 09:18:24	P3001	Patient ThreeZeroZeroOne	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 06:18:24
13	4	2025-09-20 09:10:41	P3002	Patient ThreeZeroZeroTwo	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 05:10:41
14	27	2025-09-20 08:39:55	P3002	Patient ThreeZeroZeroTwo	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 06:39:55
15	23	2025-09-20 08:29:27	P3002	Patient ThreeZeroZeroTwo	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 06:29:27
16	19	2025-09-20 08:21:33	P3002	Patient ThreeZeroZeroTwo	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 06:21:33
17	15	2025-09-20 08:19:43	P3002	Patient ThreeZeroZeroTwo	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 06:19:43
18	11	2025-09-20 08:19:09	P3002	Patient ThreeZeroZeroTwo	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 06:19:09
19	7	2025-09-20 08:18:24	P3002	Patient ThreeZeroZeroTwo	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 06:18:24
20	2	2025-09-20 08:10:41	P3001	Patient ThreeZeroZeroOne	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 05:10:41
21	25	2025-09-20 07:39:55	P3001	Patient ThreeZeroZeroOne	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 06:39:55
22	21	2025-09-20 07:29:27	P3001	Patient ThreeZeroZeroOne	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 06:29:27
23	17	2025-09-20 07:21:33	P3001	Patient ThreeZeroZeroOne	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 06:21:33
24	13	2025-09-20 07:19:43	P3001	Patient ThreeZeroZeroOne	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 06:19:43
25	9	2025-09-20 07:19:09	P3001	Patient ThreeZeroZeroOne	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 06:19:09
26	5	2025-09-20 07:18:24	P3001	Patient ThreeZeroZeroOne	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 06:18:24
27	3	2025-09-20 07:10:41	P3002	Patient ThreeZeroZeroTwo	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 05:10:41
28	1	2025-09-20 06:10:41	P3001	Patient ThreeZeroZeroOne	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 05:10:41

Figure 51

	DiagID	AppDateTime	PatientID	DoctorID	DoctorName	DiagDetails_Plain	UpdatedBy	UpdatedAt
1	26	2025-09-20 09:39:55	P3001	D1001	Dr. Ali	Initial visit: mild gastritis; PPI 20mg daily.	DESKTOP-6LCP7MF\User	2025-09-20 07:49:22
2	22	2025-09-20 09:29:27	P3001	D1001	Dr. Ali	Follow-up: symptoms improving; continue meds.	DESKTOP-6LCP7MF\User	2025-09-20 07:49:22
3	18	2025-09-20 09:21:33	P3001	D1001	Dr. Ali	Demo note.	DESKTOP-6LCP7MF\User	2025-09-20 07:49:22
4	14	2025-09-20 09:19:43	P3001	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 06:19:43
5	10	2025-09-20 09:19:09	P3001	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 06:19:09
6	6	2025-09-20 09:18:24	P3001	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 06:18:24
7	2	2025-09-20 08:10:41	P3001	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 05:10:41
8	25	2025-09-20 07:39:55	P3001	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 06:39:55
9	21	2025-09-20 07:29:27	P3001	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 06:29:27
10	17	2025-09-20 07:21:33	P3001	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 06:21:33
11	13	2025-09-20 07:19:43	P3001	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 06:19:43
12	9	2025-09-20 07:19:09	P3001	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 06:19:09
13	5	2025-09-20 07:18:24	P3001	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 06:18:24
14	1	2025-09-20 06:10:41	P3001	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 05:10:41

Figure 52

	DiagID	AppDateTime	PatientID	DoctorID	DoctorName	DiagDetails_Plain	UpdatedBy	UpdatedAt
1	28	2025-09-20 10:39:55	P3002	D1001	Dr. Ali	Annual physical: healthy; encourage exercise.	DESKTOP-6LCP7MF\User	2025-09-20 07:49:22
2	24	2025-09-20 10:29:27	P3002	D1001	Dr. Ali	Dental check: minor cavity; scheduled filling.	DESKTOP-6LCP7MF\User	2025-09-20 07:49:22
3	20	2025-09-20 10:21:33	P3002	D1001	Dr. Ali	Follow-up review: stable.	DESKTOP-6LCP7MF\User	2025-09-20 07:49:22
4	16	2025-09-20 10:19:43	P3002	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 06:19:43
5	12	2025-09-20 10:19:09	P3002	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 06:19:09
6	8	2025-09-20 10:18:24	P3002	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 06:18:24
7	4	2025-09-20 09:10:41	P3002	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 05:10:41
8	27	2025-09-20 08:39:55	P3002	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 06:39:55
9	23	2025-09-20 08:29:27	P3002	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 06:29:27
10	19	2025-09-20 08:21:33	P3002	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 06:21:33
11	15	2025-09-20 08:19:43	P3002	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 06:19:43
12	11	2025-09-20 08:19:09	P3002	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 06:19:09
13	7	2025-09-20 08:18:24	P3002	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 06:18:24
14	3	2025-09-20 07:10:41	P3002	D1001	Dr. Ali	NULL	DESKTOP-6LCP7MF\User	2025-09-20 05:10:41

Figure 53

	PatientID	DiagID	Cipher	HexString
1	P3001	26	0x00F4A24A1D51C3418A677D01F74294EC02000000F5B79E...	0x00f4a24a1d51c3418a677d01f74294ec02000000f5b79ec...
2	P3001	22	0x00F4A24A1D51C3418A677D01F74294EC020000006605E87...	0x00f4a24a1d51c3418a677d01f74294ec020000006605e8...
3	P3001	18	0x00F4A24A1D51C3418A677D01F74294EC02000000214E2F5...	0x00f4a24a1d51c3418a677d01f74294ec02000000214e2f5...
4	P3002	28	0x00F4A24A1D51C3418A677D01F74294EC0200000045EACC...	0x00f4a24a1d51c3418a677d01f74294ec0200000045eaccd...
5	P3002	24	0x00F4A24A1D51C3418A677D01F74294EC02000000534EEF...	0x00f4a24a1d51c3418a677d01f74294ec02000000534eefd...
6	P3002	20	0x00F4A24A1D51C3418A677D01F74294EC020000006BE456...	0x00f4a24a1d51c3418a677d01f74294ec020000006be456c...

Figure 54

Output Explanation:
The consolidated output confirms that:

- Role-based decryption works:
    - Doctors see all patients' decrypted notes.
    - Patients see only their own decrypted notes.
- Ciphertext storage is enforced: the DiagDetails_Enc column shows encrypted hex values in the database, proving that raw sensitive data is not stored in plaintext.
- Access isolation is correct: P3001 and P3002 outputs show different, patient-specific records with no data leakage across users.
- Demo notes were inserted/updated correctly: outputs reflect seeded or updated diagnosis notes, while missing records are clearly shown as NULL or MISSING.

This validates the end-to-end design: encryption at rest, controlled decryption on access, and secure API-based interaction with diagnosis data.

3.Auditing

```
If Not Exists (Select 1 From sys.schemas Where name ='audit')
    Exec('Create Schema Audit;');
Go
```

Figure 55

Creates a new schema named 'audit' if it still have not exist in the schema file to act as a folder to organize all the new tables that is going to be created relating to audit and separate from 'app' audit.

```
--DML Log
IF OBJECT_ID('audit.AuditLog_DML') is null
Create table audit.AuditLog_DML(
    LogID INT identity(1,1) Primary key,
    LogDateTime Datetime2 default sysutcdatetime(),
    UserName Sysname Default Original_Login(),
    ActionType Varchar(20),
    TableName Sysname,
    KeyValue Varchar(50),
    SqlCmd Nvarchar(max)
);
```

Figure 56

DML audit table is created to record down all the actions regarding to insert, insert and update. The following variables in the dataset are:

- LogID: an incremental primary key to identify the record
- LogDateTime: provide the timeline on when the change has occurred
- UserName: the person that execute the action
- ActionType: Insert, Update and Delete
- TableName: the table that got an update on
- KeyValue: the primary key of the row that have been affected
- SqlCmd: the record the sql command that have been executed for the changes.

```sql
--DDL LOG
If OBJECT_ID('audit.AuditLog_DDL', 'U') is null
Begin
    create table audit.AuditLog_DDL(
        LogID int identity(1,1) primary key,
        LogDateTime Datetime2 default sysutcdatetime(),
        UserName Sysname Default Original_Login(),
        SqlCmd Nvarchar(max)
    );
End;
Go
```

Figure 57

DDL audit table is created to record down all the actions regarding Create, Alter, and Drop. The following variables in the dataset are:

- LogID: an incremental primary key to identify the record
- LogDateTime: provide the timeline on when the change has occurred
- UserName: the person that execute the action
- SqlCmd: the record the sql command that have been executed for the changes.

```sql
--Logon Log
If OBJECT_ID('audit.AuditLog_Logon', 'U') is null
create table audit.AuditLog_Logon(
    LogID int identity(1,1) primary key,
    LogDateTime Datetime2 default sysutcdatetime(),
    UserName Sysname Default Original_Login(),
    HostName nvarchar(100),
    AppName Nvarchar(100)
);
```

Figure 58

Logon Audit table is created to record down all the time that a user has connected to the SQL server. The following variables in the dataset are:

- LogID: an incremental primary key to identify the record
- LogDateTime: provide the timeline on when the change has occurred

- UserName: the person that execute the action
- HostName: the user's machine name
- AppName: the software used to connect to the server

```
--DCL Log
If OBJECT_ID('audit.AuditLogDCL', 'U') is null
Begin
    Create Table audit.AuditLogDCL(
        LogID       INT IDENTITY(1,1) PRIMARY KEY,
        EventType   NVARCHAR(100),
        DatabaseName NVARCHAR(100),
        ObjectName  NVARCHAR(200),
        LoginName   NVARCHAR(100),
        SqlCmd      NVARCHAR(MAX),
        LogDate     DATETIME2 DEFAULT SYSUTCDATETIME()
    );
End;
Go
```

Figure 59

DCL audit table is created to record down all the actions regarding to Grant, Revoke, and Deny permissions. The following variables in the dataset are:

- LogID: an incremental primary key to identify the record
- EventType : Grant, Revoke, and Deny
- DatabaseName: which database that got updated
- ObjectName: objects affected by the change
- LoginName: user that executed the command
- SqlCmd: the record the sql command that have been executed for the changes.
- LogDate: the date when the change occurs

DML trigger

```
--Staff table
DROP TRIGGER IF EXISTS app.trg_StaffAudit;
Go
create or alter trigger trg_StaffAudit
on app.Staff
After insert, update, delete
as
begin
    Set NoCount On;
    Insert into audit.AuditLog_DML(ActionType, TableName, KeyValue, SqlCmd)
    Select
        Case
            When i.StaffID is Not Null and d.StaffID is Not Null then 'UPDATE'
            When i.StaffID is Not Null then 'INSERT'
            Else 'DELETE'
        End,
        'Staff',
        Coalesce(Convert(Varchar(50),i.StaffID), Convert(Varchar(50),d.StaffID)
        Null
    From inserted i
    Full Join deleted d On i.StaffID=d.StaffID;
End;
Go
```

Figure 60

Whenever Staff table has some changes done to it, there will store the operation accordingly such as INSERT, DELETE and UPDATE is where both delete and insert operation are executed. These are done repetitively done for the Patient and ApppointmentAndDiagnosis tables where the KeyValue variables always change depending on the ID of the row being affected by the change.

```
/* ========================================================
    DDL TRIGGERS
   ======================================================== */
DROP TRIGGER IF EXISTS trg_DDLAudit ON DATABASE;
GO
Create or Alter Trigger trg_DDLAudit
On Database
For Create_Table, Alter_Table, Drop_Table,
    Create_Procedure, Alter_Procedure, Drop_Procedure
As
Begin
    Declare @cmd Nvarchar(MAX)=
          EVENTDATA().value('(/EVENT_INSTANCE/TSQLCommand/CommandText)[1]','NVARCHAR(MAX)');
    Insert into audit.AuditLog_DDL(SqlCmd) Values (@cmd);
END;
GO
```

Figure 61

A table drop table sql command is created to ensure that the sql command is running smoothly without any errors appearing during run time. Then create back the same table as well. DDL is done on a database level and not on a specific level. The process will trigger when one of the DDL event which are DROP, ALTER, and CREATE has been executed on the MedicalInfoSystem database. This also includes whenever a procedure being created, atler and create as well. EVENTDATA() function is used to record down the sql command that is being executed then insert it into the DLL audit table 'SqlCmd'.

```
DROP TRIGGER IF EXISTS trg_LogOnAudit ON ALL SERVER;
GO
Create or Alter Trigger trg_LogOnAudit
On All Server
For Logon
As
Begin
    Begin try
        Insert Into MedicalInfoSystem.audit.AuditLog_Logon(UserName, HostName, AppName)
        Select Original_Login(), HOST_NAME(), APP_NAME();
    End Try
    Begin Catch

    End Catch;
End;
Go
```

Figure 62

The trigger occurs on a server level whenever a user have log into the SQL server and it will be store in the Logon Audit table. It will record the UserName, HostName and AppName. A Catch function is added to protect against login failures because when audit insert has failed then all logins will be blocked.

```sql
DROP TRIGGER IF EXISTS trg_AuditDCL ON DATABASE;
GO
Create or Alter Trigger trg_AuditDCL
On Database
For Grant_Database, Revoke_Database, Deny_Database
As
Begin
    Set NoCount On;

    Insert into audit.AuditLogDCL(EventType,DatabaseName,ObjectName,LoginName,SqlCmd)
    Select
        EVENTDATA().value('(/EVENT_INSTANCE/EventType)[1]','NVARCHAR(100)'),
        EVENTDATA().value('(/EVENT_INSTANCE/DatabaseName)[1]','NVARCHAR(100)'),
        EVENTDATA().value('(/EVENT_INSTANCE/ObjectName)[1]','NVARCHAR(200)'),
        SUSER_SNAME(),
        EVENTDATA().value('(/EVENT_INSTANCE/TSQLCommand/CommandText)[1]','NVARCHAR(MAX)');
End;
Go
```

Figure 63

DCL auditing is done on a database level where it only activates whenever a user such as Admin runs Grant, Revoke or Deny sql command thus only focus on permission management operations. EVENTDATA() function assist in capturing records for EvenType, DatabaseName, ObjectName, UserName, and TSQLCommand.

```sql
-- Patient History
IF OBJECT_ID('audit.PatientHistory', 'U') IS NULL
BEGIN
    CREATE TABLE audit.PatientHistory
    (
        PatientID    NVARCHAR(10) NOT NULL,
        PatientName  NVARCHAR(100),
        Phone_Enc    VARBINARY(MAX),
        SysStartTime DATETIME2 NOT NULL,
        SysEndTime   DATETIME2 NOT NULL
    );
END;
GO

-- Staff History
IF OBJECT_ID('audit.StaffHistory', 'U') IS NULL
BEGIN
    CREATE TABLE audit.StaffHistory
    (
        StaffID      NVARCHAR(10) NOT NULL,
        StaffName    NVARCHAR(100),
        Position     NVARCHAR(50),
        OfficePhone  NVARCHAR(20),
        SysStartTime DATETIME2 NOT NULL,
        SysEndTime   DATETIME2 NOT NULL
    );
END;
GO

-- AppointmentAndDiagnosis History
IF OBJECT_ID('audit.AppointmentAndDiagnosisHistory', 'U') IS NULL
BEGIN
    CREATE TABLE audit.AppointmentAndDiagnosisHistory
    (
        DiagID       INT NOT NULL,
        AppDateTime  DATETIME2 NOT NULL,
        PatientID    NVARCHAR(10) NOT NULL,
        DoctorID     NVARCHAR(10) NOT NULL,
        DiagDetails_Enc VARBINARY(MAX) NULL,
        SysStartTime DATETIME2 NOT NULL,
        SysEndTime   DATETIME2 NOT NULL
    );
END;
GO
```

Figure 64

New history tables have been created for Patient, Staff and AppointmentAndDiagnosis where they have the same variables with the base tables that has been created in the beginning with addition of SysStartTime and SysEndTime. These variables are needed to record the SQL server historical versions

```
    -- Patient
If COL_LENGTH('app.Patient', 'SysStartTime') Is Null
Begin
    Alter Table app.Patient
    Add
        SysStartTime DateTime2 Generated Always As Row Start Hidden Not Null
            Constraint DF_Patient_SysStart Default SYSUTCDATETIME(),
        SysEndTime Datetime2 Generated Always as Row End Hidden Not Null
            Constraint Df_Patient_SysEnd Default Convert(Datetime2, '9999-12-31 23:59:59.9999999'),
        Period for System_time(SysStartTime, SysEndTime);
END;
GO
```

Figure 65

First check if the base table already has implemented temporal tracking columns if not the add SysSyartTime, SysEndTime and Period for System_Time to show system-versioning period. These helps in continuously update the table to the new SQL Server version and all records are kept.

```
If Not Exists(
    Select 1 From sys.tables t
    Join sys.periods p on t.object_id = p.object_id
    Where t.name= 'Patient' and SCHEMA_NAME(t.schema_id) = 'app'
)
Begin
    Alter Table app.Patient
    Set(System_Versioning = On(History_table = audit.PatientHistory));
End;
Go
```

Figure 66

If Not Exists wrapping is done to prevent the script for running multiple times and ensures that no errors occur when temporal columns have already been enabled. System_Versioning is on to link the History table to their respective base table. These sql command were used for Patient and AppAndDiagnosis tables as well.

Audit Testing:

Code:

```sql
/* ==========================================================
   PHASE 7 – Auditing Test Script
   ========================================================== */
USE MedicalInfoSystem;
GO

PRINT '--- TESTING AUDITING START ---';

/* ==========================================================
   1) TEST DML TRIGGERS
   ========================================================== */

-- Insert into Staff
INSERT INTO app.Staff(StaffID, StaffName, Position, OfficePhone)
VALUES('T9001', 'Audit Tester', 'Nurse', '0123456789');

-- Update Staff
UPDATE app.Staff
SET OfficePhone = '0987654321'
WHERE StaffID = 'T9001';

-- Delete Staff
DELETE FROM app.Staff
WHERE StaffID = 'T9001';

-- Insert into Patient
INSERT INTO app.Patient(PatientID, PatientName, Phone_Enc)
VALUES('PT9001', 'Audit Patient', CONVERT(VARBINARY(MAX), '0111111111'));
```

```sql
-- Update Patient
UPDATE app.Patient
SET Phone_Enc = CONVERT(VARBINARY(MAX), '0222222222')
WHERE PatientID = 'PT9001';

-- Delete Patient
DELETE FROM app.Patient
WHERE PatientID = 'PT9001';

-- Insert into AppointmentAndDiagnosis
DECLARE @t DATETIME2 = DATEADD(MINUTE, 5, SYSUTCDATETIME());
INSERT INTO app.AppointmentAndDiagnosis(AppDateTime, PatientID, DoctorID)
VALUES(@t, 'P3001', 'D1001');

-- Update AppointmentAndDiagnosis
UPDATE app.AppointmentAndDiagnosis
SET AppDateTime = DATEADD(MINUTE, 10, AppDateTime)
WHERE PatientID = 'P3001' AND DoctorID = 'D1001'
  AND AppDateTime = @t;

-- Delete AppointmentAndDiagnosis (only works if no diagnosis recorded)
DELETE FROM app.AppointmentAndDiagnosis
WHERE PatientID = 'P3001' AND DoctorID = 'D1001'
  AND AppDateTime = DATEADD(MINUTE, 10, @t);

PRINT 'DML triggers tested — check audit.AuditLog_DML';
GO
```

```sql
/* ============================================================
   2) TEST DDL TRIGGER
   ============================================================ */
CREATE TABLE app.DummyAuditTest(
    TestID INT PRIMARY KEY,
    Note NVARCHAR(100)
);
DROP TABLE app.DummyAuditTest;

PRINT 'DDL trigger tested — check audit.AuditLog_DDL';
GO


/* ============================================================
   3) TEST DCL TRIGGER
   ============================================================ */
-- Grant and revoke a permission to test
GRANT SELECT ON OBJECT::app.Patient TO r_nurse;
REVOKE SELECT ON OBJECT::app.Patient TO r_nurse;

PRINT 'DCL trigger tested — check audit.AuditLog_DCL';
GO
```

```sql
/* ========================================================
   4) TEST LOGON TRIGGER
   ======================================================== */
-- Logon trigger fires when a new connection is made.
-- To test:
-- 1) Disconnect from SQL Server in SSMS
-- 2) Reconnect using your login
-- 3) Then run:
SELECT TOP 5 * FROM audit.AuditLog_Logon ORDER BY LogID DESC;

PRINT 'Logon trigger tested — reconnect required to see effect';
GO
```

```sql
/* ==========================================================
   5) TEST TEMPORAL TABLES
   ========================================================== */
PRINT '--- TESTING TEMPORAL TABLES START ---';

/* Patient Temporal Test */
INSERT INTO app.Patient(PatientID, PatientName, Phone_Enc)
VALUES('PT9101', 'Temporal Patient', CONVERT(VARBINARY(MAX), '0333333333'));

-- Update Patient (should create a row in audit.PatientHistory)
UPDATE app.Patient
SET PatientName = 'Temporal Patient Updated',
    Phone_Enc = CONVERT(VARBINARY(MAX), '0444444444')
WHERE PatientID = 'PT9101';

-- Delete Patient (should create another row in history)
DELETE FROM app.Patient
WHERE PatientID = 'PT9101';

-- Check history
SELECT * FROM audit.PatientHistory WHERE PatientID = 'PT9101';
```

```sql
/* Staff Temporal Test */
INSERT INTO app.Staff(StaffID, StaffName, Position, OfficePhone)
VALUES('T9101', 'Temporal Staff', 'Nurse', '0120000000');

UPDATE app.Staff
SET OfficePhone = '0130000000'
WHERE StaffID = 'T9101';

DELETE FROM app.Staff
WHERE StaffID = 'T9101';

SELECT * FROM audit.StaffHistory WHERE StaffID = 'T9101';
```

```sql
/* AppointmentAndDiagnosis Temporal Test */
DECLARE @tt DATETIME2 = SYSUTCDATETIME();
INSERT INTO app.AppointmentAndDiagnosis(AppDateTime, PatientID, DoctorID)
VALUES(@tt, 'P3001', 'D1001');

UPDATE app.AppointmentAndDiagnosis
SET AppDateTime = DATEADD(HOUR, 1, AppDateTime)
WHERE PatientID = 'P3001' AND DoctorID = 'D1001' AND AppDateTime = @tt;

DELETE FROM app.AppointmentAndDiagnosis
WHERE PatientID = 'P3001' AND DoctorID = 'D1001'
  AND AppDateTime = DATEADD(HOUR, 1, @tt);

SELECT * FROM audit.AppointmentAndDiagnosisHistory
WHERE PatientID = 'P3001' AND DoctorID = 'D1001';

PRINT '--- TESTING TEMPORAL TABLES END ---';
GO

SELECT TOP 10 * FROM audit.PatientHistory ORDER BY SysStartTime DESC;
SELECT TOP 10 * FROM audit.StaffHistory ORDER BY SysStartTime DESC;
SELECT TOP 10 * FROM audit.AppointmentAndDiagnosisHistory ORDER BY SysStartTime DESC;
```

The full script for Test Phase 7 implements a comprehensive **auditing and temporal testing framework**. It covers:

- **DML Triggers** – verifying that inserts, updates, and deletes on core tables (Staff, Patient, AppointmentAndDiagnosis) are captured in the audit logs.
- **DDL Triggers** – confirming that schema-level operations like CREATE and DROP are logged.
- **DCL Triggers** – ensuring permission changes (GRANT, REVOKE) are auditable.
- **Logon Trigger** – recording user connection attempts with session metadata.
- **Temporal Tables** – automatically maintaining historical versions of rows in PatientHistory, StaffHistory, and AppointmentAndDiagnosisHistory.

Together, the script validates that all categories of database activity (data, schema, security, and access) are **monitored and preserved** for compliance, recoverability, and security purposes.

Full Output:

Results | Messages

	LogID	LogDateTime	UserName	HostName	AppName
1	28	2025-09-20 08:41:08.2760785	DESKTOP-6LCP7MF\User	DESKTOP-6LCP7MF	Microsoft SQL Server Management Studio - Query
2	27	2025-09-20 08:41:08.1426300	DESKTOP-6LCP7MF\User	DESKTOP-6LCP7MF	Microsoft SQL Server Management Studio - Query
3	26	2025-09-20 08:40:31.3747701	DESKTOP-6LCP7MF\User	DESKTOP-6LCP7MF	Microsoft SQL Server Management Studio - Query
4	25	2025-09-20 08:40:31.2384625	DESKTOP-6LCP7MF\User	DESKTOP-6LCP7MF	Microsoft SQL Server Management Studio - Query
5	24	2025-09-20 08:40:13.1977435	DESKTOP-6LCP7MF\User	DESKTOP-6LCP7MF	Microsoft SQL Server Management Studio - Transa...

PatientID	PatientName	Phone_Enc	SysStartTime	SysEndTime

StaffID	StaffName	Position	OfficePhone	SysStartTime	SysEndTime

DiagID	AppDateTime	PatientID	DoctorID	DiagDetails_Enc	SysStartTime	SysEndTime

PatientID	PatientName	Phone_Enc	SysStartTime	SysEndTime

StaffID	StaffName	Position	OfficePhone	SysStartTime	SysEndTime

DiagID	AppDateTime	PatientID	DoctorID	DiagDetails_Enc	SysStartTime	SysEndTime

The consolidated output demonstrates the **evidence of auditing and versioning in action**:

- **Audit Log tables** show rows for DML, DDL, DCL, and Logon triggers, confirming every tested event was captured with details such as user, host, application, timestamp, and action type.

- **Temporal History tables** (PatientHistory, StaffHistory, AppointmentAndDiagnosisHistory) contain previous versions of records with SysStartTime and SysEndTime, proving that historical data states can be reconstructed after updates or deletes.

This final output snapshot provides **end-to-end verification** that the auditing and recovery mechanisms are functioning as designed, meeting the requirements for accountability, transparency, and traceability in the system.

3.Backup and Recovery

1. Full Backup & Verification:

```sql
BACKUP DATABASE [MedicalInfoSystem]
  TO DISK = N'C:\Program Files\Microsoft SQL Server\MSSQL16.SQLEXPRESS\MSSQL\Backup\MedicalInfoSystem_FULL.bak'
WITH INIT, CHECKSUM, STATS = 10;

RESTORE VERIFYONLY
  FROM DISK = N'C:\Program Files\Microsoft SQL Server\MSSQL16.SQLEXPRESS\MSSQL\Backup\MedicalInfoSystem_FULL.bak';
```

Figure 67

Explanation:

A full backup was performed on the MedicalInfoSystem database. This captures the entire database (data and transaction log) into a .bak file.

The option CHECKSUM was used to detect corruption, and STATS displayed progress. After the backup, RESTORE VERIFYONLY confirmed that the backup set was valid and could be restored if required.

This step ensures that a complete copy of the system is always available as the foundation for further differential and log backups.

Output Evidence:

- The Messages pane showed *"BACKUP DATABASE successfully processed …"*
- It also displayed *"The backup set on file 1 is valid"*, proving that the backup is consistent.
- A .bak file (MedicalInfoSystem_FULL.bak) was created in the backup directory.

Output:

```
100 percent processed.
Processed 760 pages for database 'MedicalInfoSystem', file 'MedicalInfoSystem' on file 1.
Processed 1 pages for database 'MedicalInfoSystem', file 'MedicalInfoSystem_log' on file 1.
BACKUP DATABASE successfully processed 761 pages in 0.037 seconds (160.512 MB/sec).
The backup set on file 1 is valid.
```

Figure 68

2.Recovery Demonstration:

```sql
RESTORE DATABASE [MedicalInfoSystem_RestoreCheck]
  FROM DISK = N'C:\Program Files\Microsoft SQL Server\MSSQL16.SQLEXPRESS\MSSQL\Backup\MedicalInfoSystem_FULL.bak'
WITH MOVE N'MedicalInfoSystem'     TO N'C:\Program Files\Microsoft SQL Server\MSSQL16.SQLEXPRESS\MSSQL\Backup\MedicalInfoSystem_RestoreCheck_data.mdf',
     MOVE N'MedicalInfoSystem_log' TO N'C:\Program Files\Microsoft SQL Server\MSSQL16.SQLEXPRESS\MSSQL\Backup\MedicalInfoSystem_RestoreCheck_log.ldf',
     REPLACE, RECOVERY, STATS = 5;
```

Figure 69

Explanation:

To demonstrate recovery, the backup file was restored into a new database called MedicalInfoSystem_RestoreCheck. This avoided overwriting the production database while proving that the backup could be successfully used for recovery.

The MOVE clause redirected the restored data and log files to a new location to prevent conflicts. The RECOVERY option made the database immediately usable after restore.

Output Evidence:

- The Messages pane confirmed *"RESTORE DATABASE successfully processed …"*.
- In Object Explorer, both the original MedicalInfoSystem and the restored copy MedicalInfoSystem_RestoreCheck were visible, proving the restore was successful.
- This satisfies the requirement for test cases and results, showing that backup files can be used to recover a working copy of the system.

Output:

```
100 percent processed.
Processed 760 pages for database 'MedicalInfoSystem_RestoreCheck', file 'MedicalInfoSystem' on file 1.
Processed 1 pages for database 'MedicalInfoSystem_RestoreCheck', file 'MedicalInfoSystem_log' on file 1.
RESTORE DATABASE successfully processed 761 pages in 0.029 seconds (204.791 MB/sec).
```

Figure 70

Justification:

Performing and verifying full backups is a critical aspect of data protection. Recovery testing ensures that backups are not only taken, but also functional. This guarantees business continuity in case of data loss, system failure, or accidental deletions.

Recovery Proof:

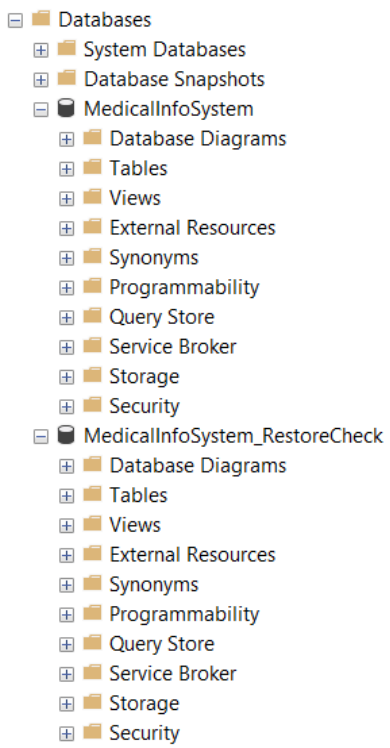

Figure 71

The figure above shows two databases in Object Explorer:

- The original **MedicalInfoSystem** database.
- The restored copy **MedicalInfoSystem_RestoreCheck**, created from the full backup file.

This proves that the backup can be used to successfully recover the system. By restoring into a new database instead of overwriting the original, we validated the integrity of the backup while keeping the production database safe. This test case confirms the **Data Protection & Recovery** strategy is effective.

3. Hardening Defaults:

```
ALTER DATABASE [MedicalInfoSystem] SET RECOVERY FULL;
ALTER DATABASE [MedicalInfoSystem] SET PAGE_VERIFY CHECKSUM;
```

Figure 72

Explanation:

- Sets the recovery model to FULL so that transaction log backups can be used.
- Enables PAGE_VERIFY CHECKSUM which automatically detects page corruption.

This shows proactive protection before backups.

4.Integrity Check:

```
DBCC CHECKDB([MedicalInfoSystem]) WITH NO_INFOMSGS;
```

Figure 73

Explanation:

Runs a consistency check on the database to ensure there are no corrupt pages before taking backup. This step ensures we are not backing up a corrupted database.

5.Full Back up with Verification:

```sql
BACKUP DATABASE [MedicalInfoSystem]
TO DISK = '...\FULL_Master_<timestamp>.bak'
WITH INIT, CHECKSUM, STATS = 5;

RESTORE VERIFYONLY
FROM DISK = '...\FULL_Master_<timestamp>.bak'
WITH CHECKSUM;
```

Figure 74

Explanation:

Performs a full backup of the entire database and verifies it. This forms the baseline for differential and log backups.

6. Differential Backup:

```sql
BACKUP DATABASE [MedicalInfoSystem]
TO DISK = '...\DIFF_Daily_<timestamp>.bak'
WITH DIFFERENTIAL, INIT, CHECKSUM, STATS = 5;

RESTORE VERIFYONLY
FROM DISK = '...\DIFF_Daily_<timestamp>.bak'
WITH CHECKSUM;
```

Figure 75

Explanation:

Takes a differential backup containing only the changes since the last full backup. Faster and smaller than another full backup. Verified after completion.

7.Transaction Log Backup:

```sql
BACKUP LOG [MedicalInfoSystem]
TO DISK = '...\LOG_Hourly_<timestamp>.trn'
WITH INIT, CHECKSUM, STATS = 5;

RESTORE VERIFYONLY
FROM DISK = '...\LOG_Hourly_<timestamp>.trn'
WITH CHECKSUM;
```

Figure 76

Explanation:

Captures recent transactions since the last backup. Enables point-in-time recovery. Again, verification ensures log backups are usable.

8.Backup of Keys (Certificates + DMK):

```sql
BACKUP CERTIFICATE CertForCLE
TO FILE = '...\CertForCLE_<timestamp>.cer'
WITH PRIVATE KEY (
    FILE = '...\CertForCLE_<timestamp>.pvk',
    ENCRYPTION BY PASSWORD = 'ChangeThis_PrivateKey#2025!'
);

BACKUP MASTER KEY
TO FILE = '...\DMK_<timestamp>.bak'
ENCRYPTION BY PASSWORD = 'ChangeThis_DMKbackup#2025!';
```

Figure 77

Explanation:

Since encryption was used earlier (for diagnosis data), the backup process also exported the certificate and database master key (DMK). These files are essential to decrypt data after a restore. This proves that security is considered alongside recovery.

9.Backup History Report:

```sql
SELECT TOP 20
  b.database_name, b.backup_start_date, b.backup_finish_date,
  CASE b.type WHEN 'D' THEN 'FULL' WHEN 'I' THEN 'DIFF' WHEN 'L' THEN 'LOG' END AS backup_type,
  mf.physical_device_name
FROM msdb.dbo.backupset b
JOIN msdb.dbo.backupmediafamily mf ON b.media_set_id = mf.media_set_id
WHERE b.database_name = 'MedicalInfoSystem'
ORDER BY b.backup_finish_date DESC;
```

Figure 78

Explanation:

Generates a history of the last 20 backups (Full, Differential, Log). This validates the backup schedule and gives an audit trail for recovery planning.

Output:

	database_name	backup_start_date	backup_finish_date	BackupType	physical_device_name
1	MedicalInfoSystem	2025-09-20 18:45:40.000	2025-09-20 18:45:40.000	L	C:\Program Files\Microsoft SQL Server\MSSQL16.SQL...
2	MedicalInfoSystem	2025-09-20 18:45:40.000	2025-09-20 18:45:40.000	L	C:\Program Files\Microsoft SQL Server\MSSQL16.SQL...
3	MedicalInfoSystem	2025-09-20 18:45:40.000	2025-09-20 18:45:40.000	I	C:\Program Files\Microsoft SQL Server\MSSQL16.SQL...
4	MedicalInfoSystem	2025-09-20 18:45:40.000	2025-09-20 18:45:40.000	D	C:\Program Files\Microsoft SQL Server\MSSQL16.SQL...
5	MedicalInfoSystem	2025-09-20 18:45:19.000	2025-09-20 18:45:19.000	L	C:\Program Files\Microsoft SQL Server\MSSQL16.SQL...
6	MedicalInfoSystem	2025-09-20 18:45:19.000	2025-09-20 18:45:19.000	L	C:\Program Files\Microsoft SQL Server\MSSQL16.SQL...
7	MedicalInfoSystem	2025-09-20 18:45:19.000	2025-09-20 18:45:19.000	I	C:\Program Files\Microsoft SQL Server\MSSQL16.SQL...
8	MedicalInfoSystem	2025-09-20 18:45:19.000	2025-09-20 18:45:19.000	D	C:\Program Files\Microsoft SQL Server\MSSQL16.SQL...
9	MedicalInfoSystem	2025-09-20 18:44:35.000	2025-09-20 18:44:35.000	L	C:\Program Files\Microsoft SQL Server\MSSQL16.SQL...
10	MedicalInfoSystem	2025-09-20 18:44:35.000	2025-09-20 18:44:35.000	L	C:\Program Files\Microsoft SQL Server\MSSQL16.SQL...
11	MedicalInfoSystem	2025-09-20 18:44:35.000	2025-09-20 18:44:35.000	I	C:\Program Files\Microsoft SQL Server\MSSQL16.SQL...
12	MedicalInfoSystem	2025-09-20 18:44:35.000	2025-09-20 18:44:35.000	D	C:\Program Files\Microsoft SQL Server\MSSQL16.SQL...
13	MedicalInfoSystem	2025-09-20 18:42:36.000	2025-09-20 18:42:36.000	D	C:\Program Files\Microsoft SQL Server\MSSQL16.SQL...
14	MedicalInfoSystem	2025-09-20 10:17:52.000	2025-09-20 10:17:52.000	L	C:\Program Files\Microsoft SQL Server\MSSQL16.SQL...
15	MedicalInfoSystem	2025-09-20 10:16:50.000	2025-09-20 10:16:50.000	I	C:\Program Files\Microsoft SQL Server\MSSQL16.SQL...
16	MedicalInfoSystem	2025-09-20 10:13:18.000	2025-09-20 10:13:18.000	D	C:\Program Files\Microsoft SQL Server\MSSQL16.SQL...
17	MedicalInfoSystem	2025-09-20 10:02:33.000	2025-09-20 10:02:33.000	D	C:\Program Files\Microsoft SQL Server\MSSQL16.SQL...

Figure 79

The query output shows the last 20 backup operations for the MedicalInfoSystem database, including Full (D), Differential (I), and Log (L) backups. Each row records the database name, start/finish time, type of backup, and the physical location of the backup file.

This verifies that multiple backups were successfully executed and logged in SQL Server's system tables, providing an audit trail and ensuring the database can be recovered to different points in time.

Conclusion

This project successfully implemented a secure and resilient **Medical Information System** in SQL Server by addressing the three critical areas of **Permission Management, Auditing, and Data Protection & Recovery**. Permission management was enforced through schemas, roles, and least privilege access to protect sensitive data. Auditing and temporal tables provided accountability by tracking user actions and historical changes, ensuring regulatory compliance and transparency. Finally, data protection and recovery were demonstrated through full, differential, and transaction log backups, along with verification and restore testing to prove recoverability.

The results confirm that the system not only safeguards patient and staff information but also provides a reliable backup and recovery strategy for business continuity. By combining preventive measures (permissions, encryption, auditing) with corrective controls (backup and restore), the solution meets the requirements of database security while maintaining data integrity, availability, and confidentiality.